signus 변경분

바뀐 파일은 통째로 실었습니다 — 그 파일 쪽만 새로 쓰면 됩니다

기준점 2026-08-24 (868c402+수정중) → 2026-08-26 · 42d9dd9+수정중 | 9개 파일 · **+181** / −415 줄 · 코드 42쪽 | 새 코드북 69쪽

바뀐 내용

· 광대역 survey 를 통째로 들어냈다 — 이제 흐름은 '버스트를 찾고 제원을 분석한다' 하나다. detect·channelize·triage 세 파일은 종이에서 지우면 되고(붉은 점선 자리), 처프/LoRa 판독은 analyze 가 이어받아 특성(처프율·대역폭·SF·심볼레이트)으로 보고한다.

· 웹은 파일을 열면 곧장 스펙트로그램 버스트 지도가 뜨고(방사체 상자 화면 삭제), 그림은 sa 프로브의 PNG 와 같은 조판(흑백, 가로=시간, 번호 레인 + 검출 띠, 백색 곡선 스펙트럼)이다. analyze 동작은 그대로다 — 복제 캡처 회신이 개조 전과 같은 #n5bt 로 확인됐다.

□	signus/dsp.py	+1	−1	2쪽	코드북 10쪽
□	signus/fsk.py	+1	−1	8쪽	코드북 25쪽
□	signus/chirp.py	+1	−1	12쪽	코드북 29쪽
□	signus/pipeline.py	+48	−123	13쪽	코드북 31쪽
□	signus/cli.py	+37	−81	21쪽	코드북 39쪽
□	signus/server.py	+3	−4	24쪽	코드북 42쪽
□	signus/web/index.html	+1	−21	25쪽	코드북 44쪽
□	signus/web/style.css	+5	−21	28쪽	코드북 47쪽
□	signus/web/app.js	+84	−162	32쪽	코드북 51쪽
□	~~signus/channelize.py — 삭제됨 (필사본에서 지운다)~~				
□	~~signus/detect.py — 삭제됨 (필사본에서 지운다)~~				
□	~~signus/triage.py — 삭제됨 (필사본에서 지운다)~~				

초록 = 새로 쓸 줄 (오른쪽 숫자가 새 줄번호) · 색 없는 줄 = 그대로 · 붉은 점선 = 그 자리에서 몇 줄 사라짐 · ↳ = 윗줄에 붙는 이어짐 (원래 한 줄이니 붙여서 입력)

signus/dp.py +1 −1 · 코드북 10쪽 · 새로 쓸 줄 79

```
"""Receiver DSP stages for blind PSK/QAM demodulation. Vectorized; per the house
anti-shared-bug rule this imports only constellation reference data, never gen.py."""

from fractions import Fraction

import numpy as np
from scipy.ndimage import uniform_filter1d
from scipy.signal import get_window, hilbert, resample_poly, stft, welch

from ._accel import dd_carrier
from .constellations import ideal_points


def analytic(x: np.ndarray) -> np.ndarray:
    """Complex passthrough as complex128; real input -> Hilbert analytic signal. Non-finite
    samples are zeroed first: a single NaN/Inf otherwise spreads through every FFT (and Inf
    overflows on the |x|^2 the estimators take)."""
    x = np.nan_to_num(x, posinf=0.0, neginf=0.0)
    x = np.where(np.abs(x) > 1e150, 0.0, x)   # a finite spike that overflows |x|^2 is corrupt t
      oo
    if np.iscomplexobj(x):
        return x.astype(np.complex128)
    return hilbert(np.asarray(x, dtype=np.float64)).astype(np.complex128)


def _parab(ydb: np.ndarray, k: int) -> float:
    """Sub-bin peak offset from a 3-point parabola fit (inputs already in dB)."""
    if k <= 0 or k >= ydb.size - 1:
        return 0.0
    a, b, c = ydb[k - 1], ydb[k], ydb[k + 1]
    d = a - 2 * b + c
    # clamp to +-half a bin: a true peak's sub-bin offset cannot exceed that, but a near-flat
    # (d~=0) or non-max triple can explode the ratio and drive baud negative -> a resample crash
      .
    return float(np.clip(0.5 * (a - c) / d, -0.5, 0.5)) if d != 0 else 0.0


def _kmeans1d(a: np.ndarray, k: int, iters: int = 50) -> tuple[np.ndarray, np.ndarray, float]:
    """1-D k-means on quantile-seeded centers; return (centers, labels, rms spread).
    Assignment is a running min over the k centers (no (N, k) broadcast temporary);
    strict `<` keeps the lowest-index center on ties, so labels are identical to a
    broadcast argmin. Shared by the classify amplitude-ring test and the FSK level demod."""
    c = np.quantile(a, (np.arange(k) + 0.5) / k)
    lab = np.zeros(a.size, dtype=int)
    for _ in range(iters):
        d = np.abs(a - c[0])
        lab = np.zeros(a.size, dtype=int)
        for j in range(1, k):
            dj = np.abs(a - c[j])
            closer = dj < d
            lab[closer] = j
            d[closer] = dj[closer]
        nc = np.array([a[lab == j].mean() if np.any(lab == j) else c[j] for j in range(k)])
        if np.allclose(nc, c):
            break
        c = nc
    return c, lab, float(np.sqrt(np.mean((a - c[lab]) ** 2)))


```

```
58 58  # --- burst detection ------------------------------------------------------
59 59
60 60  _SPIKE_PAR = 60.0  # raw peak-to-mean power above which a candidate run is an impulse smear, no
   ↳ ↳   t a
61 61  #                   burst. Modulated bursts sit low: constant-envelope FSK ~1, RRC-shaped PSK/Q
   ↳ ↳   AM
62 62  #                   peaks ~6-12, high-order QAM tails to ~20; a single-sample impulse smeared o
   ↳ ↳   ver
63 63  #                   a ~win-wide run scores ~win (hundreds+). 60 clears every real mod with marg
   ↳ ↳   in.
64 64
65 65
66 66  def _otsu(v: np.ndarray, bins: int = 128) -> float:
67 67      """Otsu split threshold of a 1-D value distribution (log powers, column scores)."""
68 68      hist, edges = np.histogram(v, bins=bins)
69 69      centers = (edges[:-1] + edges[1:]) / 2
70 70      w = np.cumsum(hist).astype(float)
71 71      m = np.cumsum(hist * centers)
72 72      mb = m / np.where(w > 0, w, 1)
73 73      mf = (m[-1] - m) / np.where(w[-1] - w > 0, w[-1] - w, 1)
74 74      return float(centers[int(np.argmax(w * (w[-1] - w) * (mb - mf) ** 2))])
75 75
76 76
77 77  def _cell_power(x: np.ndarray, fs: float, nperseg: int = 256) -> tuple[np.ndarray, int, int]:
78 78      """Waterfall cell power for detection: (P[fbin, col], hop, nperseg). nperseg
   79 +      auto-shrinks on short records; shared core for find_bursts."""
80 80      nperseg = int(min(nperseg, max(64, 1 << int(np.log2(max(x.size // 2, 64))))))
81 81      hop = nperseg // 2
82 82      win = get_window("blackmanharris", nperseg)
83 83      _, _, z = stft(x, fs=fs, window=win, nperseg=nperseg, noverlap=nperseg - hop,
84 84                     return_onesided=False, boundary=None, padded=False)
85 85      return np.abs(z) ** 2, hop, nperseg
86 86
87 87
88 88  def find_bursts(x: np.ndarray, fs: float) -> list[tuple[int, int]]:
89 89      """Waterfall (cell-level) burst detector. Per-bin noise floors give the same
90 90      narrowband processing gain the operator's eye gets on a spectrogram (a burst at
91 91      wideband 0 dB is +12 dB per cell when it occupies 1/16 of the band); column
92 92      scores then drive the run/merge/guard machinery on the time axis.
93 93      Returns bursts in time order, or [(0, size)] when nothing stands out."""
94 94      n = x.size
95 95      pw = np.abs(x) ** 2
96 96      # A burst is a TIME-ENERGY event. A continuous signal that merely shuffles its spectrum
97 97      # (multi-tone FSK) lights different cells per column and once shattered into 78 phantom
98 98      # bursts -- but its wideband envelope is FLAT. Envelope Otsu separation measured 0.014-
99 99      # 0.052 decades on continuous FSK vs >=0.30 on every real burst regime: nothing turns
100 100   # on or off -> the whole record is the burst.
101 101   lp = np.log10(uniform_filter1d(pw, max(64, n // 1000)) + 1e-20)
102 102   e_thr = _otsu(lp)
103 103   e_lo, e_hi = lp[lp < e_thr], lp[lp >= e_thr]
104 104   if not e_lo.size or not e_hi.size or float(e_hi.mean() - e_lo.mean()) < 0.12:
105 105       return [(0, n)]
106 106   # nperseg 128: bin 7.8 kHz keeps the narrowband gain for real signal widths while the
107 107   # 64-sample hop resolves inter-burst gaps down to ~200 samples (256 could not split a
108 108   # 300-sample gap -- no column fits inside it).
109 109   P, hop, nperseg = _cell_power(x, fs, nperseg=128)
110 110   if P.shape[1] < 4:                  # too few columns to tell bursts from record
111 111       return [(0, n)]
112 112   # Per-bin floor at a LOW percentile: tracks coloured noise bin by bin and stays on the
113 113   # noise even when a bin is signal-occupied up to ~90% of the time (high-duty trains).
```

```
    # A record-filling signal owns its bins' floors entirely -> flat score -> [(0, n)].
    floor_f = np.percentile(P, 10, axis=1)[:, None]
    r = P / (floor_f + 1e-30)
    k = max(3, P.shape[0] // 16)
    sc = np.log10(np.sort(r, axis=0)[-k:].mean(axis=0) + 1e-30)  # column score (decades)
    # A single noise column can spike +0.53 decades over base -- OVERLAPPING the weakest
    # real bursts (+0.57). Like the eye, the discriminator is PERSISTENCE, not height: a
    # 3-column smooth drops the noise worst-case to 0.31-0.33 while a real burst (>=3
    # columns) keeps its level (0.56+) -- the gap the fixed bars below live in.
    sc = uniform_filter1d(sc, 3)
    # The noise base is the LOWEST MODE of the score histogram (Otsu, as the wideband
    # version proved out): fixed quantile anchors cannot serve every duty regime, and
    # spread ESTIMATES broke three ways (median blind >50% duty, two-sided MAD inflated by
    # skirts, one-sided by the score's left tail). The margins are FIXED instead: a noise
    # column's score is distribution-pinned at C = log10((ln(nbins/k)+1)/0.105) whatever
    # the noise level, so its fluctuation is a constant of (nbins, k) -- measured worst
    # 0.33 over 24 noise records vs 0.56 for the weakest keeper burst.
    thr = _otsu(sc, bins=64)
    quiet = sc[sc < thr]
    if not quiet.size:
        return [(0, n)]
    base = float(np.median(quiet))
    # No noise-quiet column at all (base far above the pinned C): a continuous signal
    # shuffling its spectrum (a 4-tone FSK read as 78 phantom bursts before this guard).
    c_noise = float(np.log10((np.log(P.shape[0] / k) + 1) / 0.105))
    if base > c_noise + 0.25:
        return [(0, n)]
    hi = base + 0.45
    lo = base + 0.25
    above_lo = sc >= lo
    dif = np.diff(above_lo.astype(np.int8))
    starts = list(np.where(dif == 1)[0] + 1)
    ends = list(np.where(dif == -1)[0] + 1)
    if above_lo[0]:
        starts.insert(0, 0)
    if above_lo[-1]:
        ends.append(sc.size)
    runs = [(s, e) for s, e in zip(starts, ends, strict=True) if (sc[s:e] >= hi).any()]
    if not runs:
        return [(0, n)]
    merged = [list(runs[0])]
    for s, e in runs[1:]:
        # A real inter-burst gap returns to the score base; a marginal-level burst dips just
        # below lo without reaching it and would otherwise shatter into fragments. Merge
        # across any valley whose median stays off the base.
        valley = sc[merged[-1][1]:s]
        if s - merged[-1][1] < 2 or float(np.median(valley)) > base + 0.12:
            merged[-1][1] = e
        else:
            merged.append([s, e])
    # Shape gate: a modulated burst has a near-flat raw envelope (peak/mean ~ a few); an
    # impulse smear is one spike over noise (ratio ~span). Reject spike-dominated candidates.
    out, spiked = [], 0
    for c0, c1 in merged:
        if c1 - c0 < 2:                  # single-column flicker (noise / impulse smear)
            continue
        s, e = max(0, c0 * hop), min(n, (c1 - 1) * hop + nperseg)
        if c1 - c0 >= 6:
            # long bursts: trim one hop of smear/quantization slack per edge. The noise
            # tail otherwise fed to the demod sits on a knife edge for dense QAM (58 extra
```

```
            # leading samples flipped a 16qam@24dB slice to 64qam lock 18). Short runs keep
            # their full span -- trimming there would eat into the burst itself.
            s, e = s + hop, e - hop
        seg = pw[s:e]
        if float(seg.max() / (seg.mean() + 1e-30)) > _SPIKE_PAR:
            spiked += e - s
            continue
        out.append((int(s), int(e)))
    # A detection must EXPLAIN the candidate mass the SPIKE gate threw away. If an impulse
    # inside the strongest burst killed its candidate, returning the survivors would hand
    # analyze() the wrong emitter -- fall back instead. Only spike kills count: the
    # single-column drops above are routine flicker filtering.
    covered = sum(e - s for s, e in out)
    if out and covered < 0.6 * (covered + spiked):
        return [(0, n)]
    # A continuous signal OWNS its bins' floors and is invisible to the cell path -- but its
    # edge transients are not, and once returned [(0, 192)] for a 64k channel whose 16qam
    # ran the whole record. If the detections cover almost none of the envelope's high-power
    # mass, the cell path missed the main event: return the ENVELOPE's high span instead
    # (not the raw record -- a dead tail fed to the demod flipped that 16qam to 64qam).
    # 0.1 keeps partial detections (a strong burst next to a lo-only sibling covers ~30%).
    if out and covered < 0.1 * int((lp >= e_thr).sum()):
        idx = np.where(lp >= e_thr)[0]
        return [(int(idx[0]), int(idx[-1]) + 1)]
    return out or [(0, n)]


def find_burst(x: np.ndarray, fs: float) -> tuple[int, int]:
    """The most energetic burst of find_bursts."""
    return max(find_bursts(x, fs), key=lambda b: float(np.sum(np.abs(x[b[0]:b[1]]) ** 2)))


# --- carrier estimation ------------------------------------------------

def est_carrier(
    x: np.ndarray, fs: float, powers: tuple[int, ...] = (2, 4, 8),
    nfft: int = 65536, blocks: int = 4,
) -> tuple[float, int, bool]:
    """Blind M-th-power carrier estimate; returns (fc, symmetry, ambiguous). No DC
    nulling: the front-end is DC-blocked, so a peak at DC is a genuine fc=0."""
    freqs = np.fft.fftshift(np.fft.fftfreq(nfft, 1 / fs))
    best_par, best_s, sym = -1.0, None, powers[0]
    for p in powers:
        xp = x ** p
        blk = xp.size // blocks
        segs = [xp[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [xp]
        acc = np.zeros(nfft)
        for s in segs:
            acc += np.abs(np.fft.fftshift(np.fft.fft(s * np.hanning(s.size), nfft)))
        spec = acc / len(segs)
        par = spec.max() / spec.mean()
        if par > best_par:
            best_par, best_s, sym = par, spec, p

    if best_s is None:  # degenerate input (all-zero / no energy): no M-th-power tone exists
        return 0.0, int(powers[0]), True
    ydb = 20 * np.log10(best_s + 1e-20)
    k = int(np.argmax(best_s))
    f_peak = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
    fc = f_peak / sym
```

```
234 234     ambiguous = abs(sym * fc) > 0.4 * fs
235 235     return float(fc), int(sym), bool(ambiguous)
236 236
237 237
238 238 def resolve_alias(x: np.ndarray, fs: float, fc: float, sym: int) -> float:
239 239     """The M-th-power tone wraps mod fs, so fc is only known mod fs/sym. Candidates tile the
240 240     whole band at spacing fs/sym; pick the one whose alias CELL holds the most signal power
241 241     (wrap-aware). Cell-energy beats a spectral centroid at the band edge, where a signal
242 242     straddling +-fs/2 corrupts the two-sided centroid (bpsk fc=0.495*fs picked fc=-5000)."""
243 243     f, pxx = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False)
244 244     p = np.maximum(pxx - np.median(pxx), 0)
245 245     # range must reach +-fs/2 for EVERY sym: |k| up to sym//2 (an old fixed range(-2,3) left
246 246     # 8psk beyond 0.3125*fs with no candidate).
247 247     cands = [fc + k * fs / sym for k in range(-(sym // 2) - 1, sym // 2 + 2)
248 248              if abs(fc + k * fs / sym) < 0.5 * fs]
249 249     half = fs / (2 * sym)                          # each candidate owns a fs/sym-wide cell
250 250     return max(cands, key=lambda c: float(p[np.abs(((f - c + fs / 2) % fs) - fs / 2) < half].sum
  ↳ ↳       ()))
251 251
252 252
253 253 def mix(x: np.ndarray, fs: float, fc: float) -> np.ndarray:
254 254     """Downconvert by fc (complex heterodyne)."""
255 255     return x * np.exp(-2j * np.pi * fc * np.arange(x.size) / fs)
256 256
257 257
258 258 # --- symbol rate + rolloff ------------------------------------------------
259 259
260 260 def est_baud(
261 261     x: np.ndarray, fs: float, lo: float | None = None, hi: float | None = None,
262 262     blocks: int = 4,
263 263 ) -> tuple[float, float]:
264 264     """Cyclostationary line at the symbol rate in |x|^2; returns (baud, confidence).
265 265     Low confidence flags a weak line (near-zero rolloff); caller decides. Fewer blocks =
266 266     a longer, stronger line (worth trying on a short burst where the default 4 is too weak)."""
267 267     u = np.abs(x) ** 2
268 268     u = u - u.mean()
269 269     blk = u.size // blocks
270 270     segs = [u[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [u]
271 271     nfft = 1 << max(16, int(np.ceil(np.log2(max(s.size for s in segs)))))
272 272     acc = np.zeros(nfft // 2 + 1)
273 273     for s in segs:
274 274         acc += np.abs(np.fft.rfft(s * np.hanning(s.size), nfft))
275 275     spec = acc / len(segs)
276 276     freqs = np.fft.rfftfreq(nfft, 1 / fs)
277 277
278 278     lo = lo if lo is not None else 0.005 * fs
279 279     hi = hi if hi is not None else 0.45 * fs
280 280     band = (freqs >= lo) & (freqs <= hi)
281 281     if not band.any():  # degenerate window (e.g. noise-derived prior): default band
282 282         band = (freqs >= 0.005 * fs) & (freqs <= 0.45 * fs)
283 283     idx = np.where(band)[0]
284 284     k = idx[int(np.argmax(spec[idx]))]
285 285     ydb = 20 * np.log10(spec + 1e-20)
286 286     baud = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
287 287     conf = spec[k] / (np.median(spec[band]) + 1e-20)
288 288     return float(baud), float(conf)
289 289
290 290
291 291 def occupied_bw(x: np.ndarray, fs: float) -> float:
292 292     """Two-sided occupied bandwidth without a baud prior: threshold 10% of the way
```

```
    from the 25th-percentile floor to the 90th-percentile in-band level, walk
    outward from DC, stop at the first >20-bin below-threshold gap (notch tolerant).
    Returns 0.0 when no band hugs DC (caller must not trust the prior then)."""
    f, pxx = welch(x, fs=fs, nperseg=min(8192, x.size), return_onesided=False)
    floor = np.percentile(pxx, 25)
    order = np.argsort(np.abs(f))
    idx = np.flatnonzero(pxx[order] > floor + 0.10 * (np.percentile(pxx, 90) - floor))
    if idx.size == 0:
        return 0.0
    gaps = np.flatnonzero(np.diff(idx) > 20)   # 틈 정지가 없으면 대역 밖 스퍼(믹싱돼
    last = idx[gaps[0]] if gaps.size else idx[-1]   # -fc 에 떨어진 LO 누설)가 bw 를 부풀린다
    return float(2 * np.abs(f[order][last]))


def est_rolloff(x: np.ndarray, fs: float, baud: float) -> float:
    """Occupied-bandwidth band-edge estimate mapped through the v1 rolloff calibration."""
    n = x.size
    nper = min(8192, max(256, n // 8))
    f, pxx = welch(x, fs=fs, nperseg=nper, return_onesided=False)
    f, pxx = np.fft.fftshift(f), np.fft.fftshift(pxx)
    af = np.abs(f)
    inband = np.median(pxx[af < 0.25 * baud])
    noise = np.median(pxx[af > 0.9 * baud])
    thr = noise + 0.10 * (inband - noise)
    band = (af > 0.3 * baud) & (af < 1.0 * baud) & (pxx > thr)
    if not band.any():
        return 0.35
    raw = 2 * af[band].max() / baud - 1
    return float(np.clip(1.5 * raw + 0.026, 0.05, 1.0))


# --- resampling + matched filter ---------------------------------------

def to_sps(x: np.ndarray, fs: float, baud: float, sps: int = 4) -> np.ndarray:
    """Rational resample to exactly sps samples/symbol."""
    r = Fraction(sps * baud / fs).limit_denominator(2000)
    return resample_poly(x, r.numerator, r.denominator)


def _rx_rrc(alpha: float, span: int, sps: int) -> np.ndarray:
    """RX matched RRC taps, unit energy, odd length; singularities at t=0 and
    |t|=1/4a patched. Written independently of gen.py's TX shaper (anti-shared-bug)."""
    m = (span * sps) // 2
    t = np.arange(-m, m + 1) / sps
    a = alpha
    num = np.sin(np.pi * t * (1 - a)) + 4 * a * t * np.cos(np.pi * t * (1 + a))
    den = np.pi * t * (1 - (4 * a * t) ** 2)
    h = np.divide(num, den, out=np.zeros_like(t), where=den != 0)
    h[t == 0] = 1 - a + 4 * a / np.pi
    if a > 0:
        s = np.isclose(np.abs(t), 1 / (4 * a))
        h[s] = a / np.sqrt(2) * ((1 + 2 / np.pi) * np.sin(np.pi / (4 * a))
                                 + (1 - 2 / np.pi) * np.cos(np.pi / (4 * a)))
    return h / np.sqrt(np.sum(h ** 2))


def matched(x: np.ndarray, sps: int, alpha: float, span: int = 10) -> np.ndarray:
    """Apply the RX matched RRC filter."""
    return np.convolve(x, _rx_rrc(alpha, span, sps), "same")

```

```
353 353
354 354 # --- timing recovery ---------------------------------------------------------
355 355
356 356 def timing(x: np.ndarray, sps: int, block: int = 256, out: int = 1) -> np.ndarray:
357 357     """Block-wise Oerder-Meyr timing recovery; returns `out` samples/symbol
358 358     (1 = decisions, 2 = clock-tracked T/2 stream for the fractionally-spaced eq).
359 359     Per block the spectral-line phase gives the offset; unwrapped across blocks
360 360     to track clock drift, then interpolated per symbol."""
361 361     n = sps
362 362     nsym = x.size // n
363 363     if nsym < 1:
364 364         return x[:0].astype(np.complex128)
365 365     xt = x[:nsym * n]
366 366     p = np.abs(xt) ** 2
367 367     ph = np.exp(-2j * np.pi * np.arange(xt.size) / n)
368 368     bs = block * n
369 369     if xt.size <= bs:  # single block: one global fractional offset
370 370         eps = np.full(nsym, -np.angle(np.sum(p * ph)) / (2 * np.pi))
371 371     else:
372 372         nblk = xt.size // bs
373 373         c = (p[:nblk * bs].reshape(nblk, bs) * ph[:nblk * bs].reshape(nblk, bs)).sum(1)
374 374         eps_b = np.unwrap(-np.angle(c)) / (2 * np.pi)
375 375         eps = np.interp(np.arange(nsym), (np.arange(nblk) + 0.5) * block, eps_b)
376 376     pos = (np.arange(nsym * out) // out + np.repeat(eps, out)) * n
377 377     idx = np.arange(xt.size)
378 378     return np.interp(pos, idx, xt.real) + 1j * np.interp(pos, idx, xt.imag)
379 379
380 380
381 381 # --- decision-directed carrier sync -----------------------------------------
382 382
383 383 def ddsync(symbols: np.ndarray, mod: str, alpha: float = 0.05, beta: float = 0.002) -> np.ndarra
    ↳ ↳   y:
384 384     """Decision-directed PI carrier loop, general across PSK/QAM (never a PSK-only
385 385     Costas loop, which jitters QAM at the correct points). Sequential feedback (each
386 386     phase update depends on the prior decision) -> numba kernel when available."""
387 387     pts = ideal_points(mod)
388 388     z = symbols / np.sqrt(np.mean(np.abs(symbols) ** 2))  # AGC to unit power
389 389     return dd_carrier(np.ascontiguousarray(z, dtype=np.complex128),
390 390                       np.ascontiguousarray(pts, dtype=np.complex128), alpha, beta)
```

signus/fsk.py +1 −1 · **코드북 25쪽** · 새로 쓸 줄 134

```
1  1  """FSK/CPM receive path: constant-envelope gate + frequency-discriminator demod.
2  2  Runs on the DC-blocked analytic burst, before any carrier work (linear front-end
3  3  does not apply). Imports only constellation reference data (anti-shared-bug rule)."""
4  4
5  5  import numpy as np
6  6  from scipy.ndimage import uniform_filter1d
7  7  from scipy.signal import firwin, resample_poly, welch
8  8
9  9  from .constellations import fsk_levels, to_bits
10 10 from .dsp import _kmeans1d, _parab, analytic, mix
11 11
12 12 _CV_MAX = 0.24    # envelope coeff-of-variation ceiling. Recalibrated on a broad grid: real
13 13 # FSK (all h x SNR>=10 x high baud) tops out at cv 0.224, but constant-modulus PSK at very
14 14 # high baud (fs/baud~3) or near +-fs/2 dips to cv>=0.250 and used to slip the old 0.30 gate
15 15 # (false FSK). 0.24 sits in the 0.026 gap -> keeps every real FSK, rejects the PSK misfires.
16 16 _SEP_MIN = 3.1    # instantaneous-freq 2-means separation floor (FSK>=3.39, rest<=2.85)
17 17 _K4_RATIO = 2.6   # sp(k=2)/sp(k=4) collapse above which 4 levels beat 2
18 18 _MIN_SYMS = 8     # fewer symbols than this cannot cluster 2/4 levels (empty-quantile crash)
19 19 _PROM_MIN = 10.0  # a symbol-rate line peak/median below this is unreliable -- BUT only confiden
```

```
  t-
#                   wrong when paired with too few symbols (a long low-index burst has a weak li
  ne
#                   yet a correct baud). The sweep sits at prom 23-73.
_NSYM_MIN = 200   # ...so reject a weak line only below this symbol count. A 100 floor was tried
#                   (to recover more short bursts) but an independent grid found weak-prominence
#                   half-rate folds in the 100-199 band that still cluster tight enough to loc
  k >=60
#                   (msk h=0.5 baud 24-32% off, lock 61-65) -- confident-wrong. 200 keeps them o
  ut;
#                   precision over recall (the short-burst confident-wrong is the cardinal sin).
_DECIM_FRAC = 0.30  # decimate a heavily-oversampled burst so the tones fill ~this of Nyquist
_DECIM_MIN = 3      # ...but only when that needs a factor >= this, so the sps~10 demod grid
#                     (fsk2/fsk4/msk baud=fs/10) is untouched -> the sweep stays byte-identical


def _bw99(x: np.ndarray, fs: float) -> float:
    """Two-sided 99%-power occupied bandwidth (Hz), noise-pedestal subtracted."""
    f, p = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False, detrend=False)
    f, p = np.fft.fftshift(f), np.fft.fftshift(p)
    p = np.maximum(p - np.median(p), 0.0)
    cs = np.cumsum(p) / (p.sum() + 1e-30)
    lo = f[min(np.searchsorted(cs, 0.005), f.size - 1)]  # clamp BOTH edges: a degenerate PSD
    hi = f[min(np.searchsorted(cs, 0.995), f.size - 1)]  # (all-zero after floor subtraction)
    return float(abs(hi - lo))                           # otherwise indexes one past the end


def _dcblock(x: np.ndarray) -> np.ndarray:
    """Complex128 analytic burst, DC-blocked (front-end convention)."""
    x = analytic(x)
    return x - x.mean()


def _ifreq(x: np.ndarray, fs: float) -> np.ndarray:
    """Instantaneous frequency (Hz) from sample-to-sample phase differences."""
    return np.angle(x[1:] * np.conj(x[:-1])) * fs / (2 * np.pi)


def _sep(a: np.ndarray) -> float:
    """2-means between/within separation ratio of a 1-D sample set."""
    c, _, sp = _kmeans1d(a, 2)
    return abs(c[1] - c[0]) / (sp + 1e-9)


def _est_baud(f: np.ndarray, fs: float) -> tuple[float, float]:
    """Symbol rate from the spectral line of |diff(smoothed f)| (transition impulses),
    harmonic-folded to the fundamental (2-level lines alias to 2*baud). Returns
    (baud, prominence) where prominence = peak/median of the in-band line: a weak line
    (few symbols, no clean symbol clock) means the baud is UNRELIABLE -> caller rejects."""
    d = np.abs(np.diff(uniform_filter1d(f, 2)))
    d = d - d.mean()
    n = 1 << int(np.ceil(np.log2(d.size)))
    spec = np.abs(np.fft.rfft(d * np.hanning(d.size), n))
    freqs = np.fft.rfftfreq(n, 1 / fs)
    df = freqs[1] - freqs[0]
    lo, hi = 0.002 * fs, 0.45 * fs
    idx = np.where((freqs >= lo) & (freqs <= hi))[0]
    prom = float(spec[idx].max() / (np.median(spec[idx]) + 1e-20)) if idx.size else 0.0
    k = idx[int(np.argmax(spec[idx]))]

```

```
    def _hp(kk: int) -> float:                          # harmonic-comb energy: the TRUE fundamental
        return float(sum(spec[m * kk] for m in range(1, 6) if m * kk < spec.size))  # captures most
    #   in-range harmonics ONLY: clamping out-of-range ones to the last bin multi-counted the
    #   Nyquist bin, inflating _hp at a 2*baud peak enough to veto the legitimate fold -> 2x baud.

    for div in (3, 2):
        ks = int(round(freqs[k] / div / df))
        if freqs[ks] >= lo:
            w = spec[max(0, ks - 2):ks + 3]
            kk = max(0, ks - 2) + int(np.argmax(w)) if w.size else k
            # fold to the sub-harmonic ONLY if it is genuinely the fundamental -- its harmonic comb
            # must beat the current peak's. A real baud/2 fundamental captures MORE harmonic lines
            # than the 2*baud peak; spurious low-h/low-SNR energy at baud/2 does not (it once
            # slipped the 0.34 gate and folded baud -> baud/2: a confident WRONG decode). The comb
            # guard vetoes that. On the sps~10 grid the fold never fired (0.34 unmet) -> sweep same.
            if w.size and w.max() > 0.34 * spec[k] and _hp(kk) >= _hp(k):
                k = kk
    return float(freqs[k] + _parab(20 * np.log10(spec + 1e-20), k) * df), prom


def fsk_gate(x: np.ndarray, fs: float) -> bool:
    """True when the burst looks like FSK/CPM: near-constant envelope AND a bimodal
    instantaneous frequency. Calibrated on generate() over all GEN_MODS x seeds 0..3
    x snr {25,15}: every linear mod fails one test with margin -- bpsk/dbpsk have
    cv~0.40 (>_CV_MAX), the rest have freq-sep<=2.85 (<_SEP_MIN); fsk2/fsk4/msk keep
    cv<=0.22 and freq-sep>=3.39 down to snr 10 (margins ~0.08 and ~0.29).
    Recenter by the mean instantaneous frequency first: at a large carrier offset a
    linear mod's phase-transition spikes wrap past +-fs/2 and fake a second mode."""
    x = _dcblock(x)
    a = np.abs(x)
    if a.std() / (a.mean() + 1e-12) >= _CV_MAX:
        return False
    f = _ifreq(x, fs)
    x = x * np.exp(-2j * np.pi * np.mean(f) / fs * np.arange(x.size))
    return bool(_sep(uniform_filter1d(_ifreq(x, fs), 4)) > _SEP_MIN)


def analyze_fsk(x: np.ndarray, fs: float) -> dict:
    """Full FSK/MSK demod by frequency discrimination. Returns mod, fc, baud, h,
    lock (0-100), level_freqs (Hz, sorted, centered), symbols (normalized f per
    symbol) and Gray-demapped bits."""
    x = _dcblock(x)
    f = _ifreq(x, fs)
    # A carrier OFFSET (tones sit at fc +- dev, not around DC) corrupts the symbol-rate estimate two
    # ways -- off-centre it trips _est_baud's harmonic fold, and the tones fall outside the decim
    # LPF passband -- either way a confident WRONG baud. Recentre the discriminator on the carrier
    # (mean instantaneous freq) first; only for a significant offset, so a centred burst
    # (fc~0, the whole demod/sweep grid) is byte-identical. Levels are already centred downstream.
    fc0 = float(np.mean(f))
```

```
125 125     if abs(fc0) > 0.005 * fs:
126 126         x = mix(x, fs, fc0)
127 127         f = _ifreq(x, fs)
128 128     else:
129 129         fc0 = 0.0
130 130     baud, prom = _est_baud(f, fs)
131 131     # heavy oversampling (fs/baud >> 10) buries the symbol-rate line under broadband transition
132 132     # artifacts -> _est_baud locks a spurious peak (baud ~25% off) while the clean tones still
133 133     # cluster (lock ~94): a confident WRONG decode. Re-estimate baud on a decimated copy (tone
    ↳ ↳       s ->
134 +     # ~_DECIM_FRAC of Nyquist). Gated at _DECIM_MIN so the sps~10
135 135     # grid is untouched (fsk2/fsk4/msk baud=fs/10 -> d<3 -> no re-estimate -> sweep byte-identic
    ↳ ↳       al).
136 136     bw = _bw99(x, fs)
137 137     d = int(_DECIM_FRAC * fs / bw) if bw > 0 else 1
138 138     if d >= _DECIM_MIN:
139 139         xd = np.convolve(x, firwin(129, min(0.9 * (fs / d) / 2, bw) / (fs / 2)), "same")
140 140         baud, prom = _est_baud(_ifreq(resample_poly(xd, 1, d), fs / d), fs / d)
141 141     if baud <= 0:
142 142         raise ValueError("FSK 버스트에서 심볼율을 추정할 수 없습니다 (너무 짧음)")
143 143     sps = fs / baud
144 144     f = uniform_filter1d(f, max(1, int(round(sps / 2))))
145 145     resid = float(f.mean())
146 146     fc = fc0 + resid                    # absolute carrier = recentre offset + in-band residual
147 147     fcen = f - resid
148 148     nsym = int(fcen.size / sps) - 1
149 149     # a WEAK symbol-rate line (prom < _PROM_MIN) is unreliable ONLY when symbols are also too fe
    ↳ ↳       w to
150 150     # average (nsym < _NSYM_MIN): a short burst's spurious peak still clusters tight (few point
    ↳ ↳       s ->
151 151     # high lock) = a confident WRONG baud. A long low-index (h~0.35) burst also has a weak lin
    ↳ ↳       e but
152 152     # MANY symbols -> its baud is correct -> NOT rejected. Sweep: nsym~3000/prom 23-73 -> untouc
    ↳ ↳       hed.
153 153     if nsym < _MIN_SYMS or (prom < _PROM_MIN and nsym < _NSYM_MIN):
154 154         raise ValueError(f"FSK 버스트가 너무 짧거나 심볼율이 불확실합니다 ({nsym} 심볼)")
155 155     ctr = (np.arange(nsym) + 0.5) * sps
156 156
157 157     # symbol sampling phase: the offset whose sampled levels separate best
158 158     v, best = fcen[:nsym], -1.0
159 159     for o in np.linspace(0, sps, 8, endpoint=False):
160 160         s = fcen[np.clip((ctr + o).astype(int), 0, fcen.size - 1)]
161 161         sep = _sep(s)
162 162         if sep > best:
163 163             best, v = sep, s
164 164
165 165     # level count via spread-collapse (k=2 vs k=4), then cluster
166 166     k = 4 if _kmeans1d(v, 2)[2] / (_kmeans1d(v, 4)[2] + 1e-9) > _K4_RATIO else 2
167 167     c, lab, spread = _kmeans1d(v, k)
168 168     order = np.argsort(c)
169 169     ranks = np.argsort(order)[lab]          # per-symbol level index, 0 = lowest freq
170 170     csort = c[order]
171 171     mid = float(csort.mean())               # debias center: symmetric levels sum ~0
172 172     fc, csort, v = fc + mid, csort - mid, v - mid
173 173     spacing = float(np.mean(np.diff(csort)))
174 174     h = spacing / baud
175 175
176 176     mod = "msk" if (k == 2 and abs(h - 0.5) < 0.15) else f"fsk{k}"
177 177     _, glabels = fsk_levels(mod)
178 178     bits = to_bits(glabels[ranks], mod).astype(np.uint8)
```

```
179 179     lock = float(np.clip((spacing / (spread + 1e-9) - 1.5) / 6.5 * 100, 0, 100))
180 180     return dict(mod=mod, fc=fc, baud=baud, h=h, lock=lock, level_freqs=csort,
181 181                 symbols=v / (spacing / 2), bits=bits)
```

signus/chirp.py +1 −1 · **코드북 29쪽** · 새로 쓸 줄 57

```
1  1  """Blind linear-chirp / CSS (LoRa) detection + characterization. A constant-envelope
2  2  signal whose instantaneous frequency ramps linearly de-chirps to a tone: the delay-
3  3  conjugate x[t]*conj(x[t-tau]) is a beat tone at mu*tau (NONZERO), unlike FSK/CW (beat at
4  4  DC), PSK/QAM (not constant-envelope), or FM-voice/noise (broadband). Characterize-only --
5  5  reported like analog/tone, NEVER demodulated: blind CSS symbol decode needs preamble
6  6  CFO/STO sync plus LoRa's reverse-engineered Gray/interleave/Hamming/whitening framing.
7  7  Calibrated on gen chirps vs every other family: chirp beat-PAR>=936, non-chirp<=264."""
8  8
9  9  import numpy as np
10 10 from scipy.ndimage import uniform_filter1d
11 11 from scipy.signal import welch
12 12
13 13 _PAR_MIN = 400.0   # delay-conjugate beat-tone peak-to-mean floor (chirp>=936, non-chirp<=264)
14 14
15 15
16 16 def _bw99(x: np.ndarray, fs: float) -> float:
17 17     """99%-power occupied bandwidth (a chirp fills its band ~flat). Floor-subtracted so the
18 18     noise pedestal in the channel's LPF passband does not inflate the width."""
19 19     f, p = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False, detrend=False)
20 20     f, p = np.fft.fftshift(f), np.fft.fftshift(p)
21 21     p = np.maximum(p - np.median(p), 0.0)    # remove the noise pedestal
22 22     cs = np.cumsum(p) / (p.sum() + 1e-30)
23 23     lo = f[min(np.searchsorted(cs, 0.005), f.size - 1)]  # clamp BOTH edges: a degenerate PSD
24 24     hi = f[min(np.searchsorted(cs, 0.995), f.size - 1)]  # (all-zero after floor subtraction)
25 25     return float(abs(hi - lo))                           # otherwise indexes one past the end
26 26
27 27
28 28 def _beat(x: np.ndarray, fs: float) -> tuple[float, float]:
29 29     """(peak-to-mean, mu[Hz/s]) of the DC-nulled delay-conjugate spectrum."""
30 30     x = x - x.mean()
31 31     tau = max(1, x.size // 200)
32 32     y = x[tau:] * np.conj(x[:-tau])
33 33     nf = 1 << int(np.ceil(np.log2(y.size)))
34 34     spec = np.abs(np.fft.fftshift(np.fft.fft((y - y.mean()) * np.hanning(y.size), nf))) ** 2
35 35     fr = np.fft.fftshift(np.fft.fftfreq(nf, 1 / fs))
36 36     spec[np.abs(fr) < 0.002 * fs] = 0.0    # null DC band: FSK/CW/tone beat here, chirps do not
37 37     k = int(np.argmax(spec))
38 38     return float(spec[k] / (spec.mean() + 1e-30)), float(fr[k] * fs / tau)
39 39
40 40
41 41 def is_chirp(x: np.ndarray, fs: float) -> bool:
42 42     """True when a channel is a linear chirp / CSS (LoRa/FMCW). The beat-tone PAR is the
43 43     discriminator: a huge margin (chirp>=936 vs PSK/QAM<=94, FSK<=91, FM<=264, noise<=13)
44 44     makes an envelope-CV pre-gate redundant AND harmful (it rejects noisy-but-clear chirps)."""
45 45     if x.size < 256:
46 46         return False
47 47     return _beat(x, fs)[0] >= _PAR_MIN
48 48
49 49
50 50 _SWEEP_PM_MAX = 1.45   # instantaneous-freq histogram peak-to-mean: a band-sweep stays ~1
51 51 _SWEEP_OCC_MIN = 0.6   # ...and fills its band; FSK sits on 2-4 discrete tones (peaky, sparse)
52 52
53 53
54 54 def sweeps_band(x: np.ndarray, fs: float, bins: int = 40) -> bool:
55 55     """True when the instantaneous frequency SWEEPS the channel (linear chirp / CSS) rather
```

```
    than hopping between a few discrete FSK tones. A linear FMCW ramp reads bimodal to
+   fsk_gate AND trips is_chirp's beat test, so analyze's chirp gate needs this to tell a sweep
    from FSK: a sweep's IF histogram is flat and fully occupied; FSK's spikes at its tones.
    Calibrated on extracted channels: 0/222 FSK/MSK (snr 8-40) pass, 214/216 FMCW + 60/60 LoRa
    pass (misses only the narrowest, gentlest ramps -- which stay 'fsk', never a regression).
    BOTH the raw and a lightly-smoothed IF must look like a sweep: at moderate SNR (~14) noise
    flattens integer-h FSK's raw histogram to sweep-like pm ~1.44 (knife-edge), but smoothing
    restores its tone spikes (pm 2.5+) while a genuine ramp stays flat (pm ~1.1) -- and at LOW
    SNR smoothing can flatten FSK instead, which the raw view still catches. The conjunction
    only ever narrows 'sweep', so every calibrated chirp above keeps its label (margin >=0.3)."""
    if x.size < 256:
        return False
    x = x - x.mean()
    f0 = np.diff(np.unwrap(np.angle(x))) * fs / (2 * np.pi)
    for f in (f0, uniform_filter1d(f0, 4)):
        lo, hi = np.percentile(f, 1), np.percentile(f, 99)
        if hi - lo < 1:
            return False
        fv = f[(f >= lo) & (f <= hi)]
        h = np.histogram(fv, bins=bins)[0] / fv.size
        pm = h.max() / (h.mean() + 1e-30)          # discrete tones spike this; a sweep ~1
        occ = float((h > 0.005).mean())            # a sweep fills the band; tones occupy few b
            ins
        if not (pm < _SWEEP_PM_MAX and occ >= _SWEEP_OCC_MIN):
            return False
    return True


def analyze_chirp(x: np.ndarray, fs: float) -> dict:
    """Characterize a chirp: {mu[Hz/s], up, bw, par, sf, rs, tsym}. A LoRa hypothesis
    (sf, symbol rate, symbol time) is filled when bw^2/|mu| snaps to 2^(7..12); otherwise
    it stays a generic linear chirp / FMCW. bw is measured on the channel, not asserted."""
    par, mu = _beat(x, fs)
    bw = _bw99(x, fs)
    sf = rs = tsym = None
    if mu != 0 and bw > 0:
        r = float(np.log2(bw * bw / abs(mu)))
        if 7 <= round(r) <= 12 and abs(r - round(r)) < 0.3:    # power-of-two chip count -> LoRa
            sf = int(round(r))
            rs = abs(mu) / bw
            tsym = 2 ** sf / bw
    return {"mu": mu, "up": bool(mu > 0), "bw": float(bw),
            "par": par, "sf": sf, "rs": rs, "tsym": tsym}
```

signus/pipeline.py +48 −123 · **코드북 31쪽** · 새로 쓸 줄 13, 16-17, 57, 83, 103-106, 226-250, 409, 411-423

```
"""End-to-end blind demodulation. Two families:
  linear (PSK/QAM): front-end -> carrier -> baud -> matched filter -> timing ->
                    classify -> fine sync -> [equalizer rescue] -> quality
  fsk (CPFSK/MSK):  frequency discriminator (gated by constant envelope)
Classification runs BEFORE fine sync so the decision-directed loop knows the
constellation. Differential demapping is an option, not a detection: dqpsk and
qpsk share a constellation."""

import json
from dataclasses import dataclass, field

import numpy as np
+ from scipy.signal import firwin, resample_poly

```

```
14 15  from . import classify as cl
   16 + from . import dsp
   17 + from .chirp import _bw99, analyze_chirp, is_chirp, sweeps_band
18 18  from .constellations import demap_bits, demap_diff_bits, mod_order
20 19  from .eq import equalize, equalize_fse
21 20  from .fsk import analyze_fsk, fsk_gate
22 21  from .sigio import Meta, parse_name, read
23 22  from .spectrum import spectrum, waterfall
24 23  from .sync import find_preamble
25 24
26 25  _BITS_CAP = 65536
27 26  _EQ_LOCK = 60.0    # below this a linear signal is worth an equalizer attempt
28 27  _EQ_GAIN = 3.0     # ...keep it only if lock improves by this much
29 28  _EQ_FLOOR = 50.0   # ...and clears this floor, so a wrong-mod fit is never locked in
30 29  _EQ_QAM_FLOOR = 60.0  # ...but a DENSE-QAM (32/64) eq fit must clear a higher bar: the DD loop c
   an
31 30  #                       drag a lower-order cloud onto a 32/64 lattice at a marginal lock (~51)
   , a
32 31  #                       confident WRONG order. Genuine multipath recoveries clear ~72; the sweep
   's
33 32  #                       eq cases are qpsk/16qam (never 32/64 via eq) so this bar is byte-identic
   al.
34 33  _BAUD_GAIN = 5.0   # an in-band baud hypothesis must beat the global one by this lock
35 34  _SHORT_LOCK = 60.0  # below this, retry the baud line with fewer blocks (short-burst rescue)
36 35  _SYNC_LOCK = 60.0   # below this, try a repeated-preamble carrier/timing sync (packetized bursts
   )
37 36  _ALIAS_MARGIN = 10.0  # a preamble carrier alias must beat the next by this lock, else ambiguous
   .
38 37  #                       This is the LOAD-BEARING rotation guard: even when the coarse anchor itse
   lf
39 38  #                       aliases and validates a rotation at ~80 lock, that rotation wins by a thi
   n
40 39  #                       margin (~8) -- so a 10-lock margin refuses it. Because the margin catche
   s it,
41 40  #                       the lock floor can stay modest (78, not 82) -> ~5% more genuine recoverie
   s.
42 41  _SYNC_ACCEPT = 78.0  # ...AND clear this absolute lock: below it correct/rotated aliases overlap
   . A
43 42  #                       65 floor was tried (to recover more moderate bursts) but an independent
44 43  #                       snr-12 grid found 8psk rotated aliases locking 71-72 in the opened 65-7
   8 band
45 44  #                       (carrier off by one baud/L step -> confident garbage bits). Even 78 leak
   s a
46 45  #                       a few hard-corner rotations (a separate pre-existing gate limit): hold a
   t 78.
47 46  _SYNC_ADIST = 1.5    # ...AND sit within this many alias steps of the coarse est_carrier fc -- a
48 47  #                       soft backstop to the margin: it catches the one rotation the margin misse
   s,
49 48  #                       but tuned LOOSE (1.5, not 0.75) so it does not needlessly reject genuine
50 49  #                       large-offset recoveries. The three together give 0 confident-wrong across
51 50  #                       1920 hard-corner bursts at the max recall this gate reaches.
52 51  _DIFF_OF = {"bpsk": "dbpsk", "qpsk": "dqpsk"}  # pi4dqpsk arrives as qpsk -> dqpsk
53 52
54 53
55 54  @dataclass
56 55  class Result:
57 56      meta: Meta
   57 +     family: str                    # 'linear' | 'fsk' | 'chirp'
59 58      burst: tuple[int, int]
60 59      fc: float
```

```
61 60      baud: float
62 61      mod: str
63 62      lock: float
64 63      symbols: np.ndarray            # aligned symbols (linear) / normalized levels (fsk)
65 64      bits: np.ndarray
66 65      iq_corr: np.ndarray            # exportable corrected baseband
67 66      burst_x: np.ndarray = field(repr=False, default=None)  # for spectrum views
68 67      symmetry: int = 0
69 68      carrier_ambiguous: bool = False
70 69      baud_conf: float = 0.0
71 70      rolloff: float | None = None
72 71      h: float | None = None         # FSK modulation index
73 72      evm: float = float("nan")
74 73      mer_db: float = float("nan")
75 74      occupied: int = 0
76 75      snr_est: float = float("nan")
77 76      eq_applied: bool = False
78 77      eq_mode: str | None = None     # 'sym' | 'fse'
79 78      alias_resolved: bool = False
80 79      baud_fallback: bool = False
81 80      bursts: list = field(default_factory=list)
82 81      burst_idx: int = 0
83 82      preamble: object = None        # sync.Preamble when a repeated preamble drove the decode
   83 +    chirp: dict | None = None      # chirp/CSS characterization (family 'chirp': no symbols)
84 84
85 85      def to_json(self, max_points: int = 6000, views: bool = True) -> dict:
86 86          z = np.nan_to_num(self.symbols)   # a dead/DC capture -> NaN symbols -> invalid JSON
87 87          if z.size > max_points:  # keep TIME ORDER: the UI animates this sequence
88 88              z = z[np.linspace(0, z.size - 1, max_points).astype(int)]
89 89          det = {"fc": round(self.fc, 3), "baud": round(self.baud, 3), "mod": self.mod,
90 90                 "symmetry": self.symmetry, "carrier_ambiguous": self.carrier_ambiguous,
91 91                 "baud_conf": round(self.baud_conf, 1),
92 92                 "alias_resolved": self.alias_resolved,
93 93                 "baud_fallback": self.baud_fallback}
94 94          det["rolloff"] = None if self.rolloff is None else round(self.rolloff, 3)
95 95          rf = self.meta.rf_center                      # real RF = capture centre + baseband fc
96 96          det["rf_hz"] = None if rf is None else round(rf + self.fc, 3)
97 97          if self.h is not None:
98 98              det["h"] = round(self.h, 3)
99 99          if self.preamble is not None:                 # a repeated preamble drove the sync
100 100             p = self.preamble
101 101             det["preamble"] = {"period": p.period, "cfo_hz": round(p.cfo_hz, 1),
102 102                                "conf": round(p.conf, 2), "start": p.start, "end": p.end}
   103 +        if self.chirp is not None:      # chirp/CSS characterization -- no constellation fields
   104 +            c = self.chirp
   105 +            det["chirp"] = {"mu": _r(c["mu"], 1), "up": bool(c["up"]), "bw": _r(c["bw"], 1),
   106 +                            "sf": c["sf"], "rs": _r(c["rs"], 1), "tsym": _r(c["tsym"], 6)}
103 107         doc = {
104 108             "fs": self.meta.fs, "fmt": self.meta.fmt, "dtype": self.meta.dtype,  # dtyp
    ↳ ↳            e 은 한 줄
105 109             "rf_center": self.meta.rf_center,          # 보고에 필수: lock 0 의 1순위 용의자다
106 110
107 111             "family": self.family, "n_samples": int(self.iq_corr.size),
108 112             "burst": {"start": self.burst[0], "end": self.burst[1]},
109 113             "bursts": [{"start": s, "end": e} for s, e in self.bursts],
110 114             "burst_idx": self.burst_idx,
111 115             "detected": det,
112 116             "quality": {"lock": _r(self.lock, 1) or 0.0, "mer_db": _r(self.mer_db),
113 117                         "evm": _r(self.evm, 4), "occupied": int(self.occupied)},
114 118             "snr_est_db": _r(self.snr_est),
```

```
            "eq": {"applied": self.eq_applied, "mode": self.eq_mode},
            "constellation": {"i": np.round(z.real, 4).tolist(),
                              "q": np.round(z.imag, 4).tolist()},
            "bits": "".join(map(str, self.bits[:_BITS_CAP])),
        }
        if views and self.burst_x is not None:
            doc["spectrum"] = spectrum(self.burst_x, self.meta.fs)
            doc["waterfall"] = waterfall(self.burst_x, self.meta.fs)
        return doc

    def save_iq(self, path: str) -> None:
        np.column_stack([self.iq_corr.real, self.iq_corr.imag]).astype("<f4").tofile(path)

    def save_symbols(self, path: str) -> None:
        np.save(path, self.symbols.astype(np.complex64))

    def save_bits(self, path: str, packed: bool = False) -> None:
        if packed:
            np.packbits(self.bits).tofile(path)
        else:
            with open(path, "w") as fh:
                fh.write("".join(map(str, self.bits)))

    def save_report(self, path: str) -> None:
        with open(path, "w") as fh:
            json.dump(self.to_json(), fh, indent=1)


def _r(v: float, nd: int = 2) -> float | None:
    return None if v is None or not np.isfinite(v) else round(float(v), nd)


def _fine(z: np.ndarray, mod: str) -> np.ndarray:
    """Bulk-phase removal, decision-directed carrier loop, final rotation lock."""
    return cl.align(dsp.ddsync(cl.align(z, mod), mod), mod)


def _blocks(n: int) -> int:
    """M-th-power spectra are averaged INCOHERENTLY, so a weak tone (high symmetry,
    low SNR) wants fewer, longer segments; only long records need more for
    stationarity. Worth ~2 dB at the 8PSK edge over a fixed blocks=4."""
    return int(np.clip(n // 30000, 1, 8))


def _demod(xd: np.ndarray, fs: float, baud: float, symmetry: int) -> tuple:
    """Matched filter + timing + classify + fine sync at one baud hypothesis."""
    alpha = dsp.est_rolloff(xd, fs, baud)
    ym = dsp.matched(dsp.to_sps(xd, fs, baud), 4, alpha)
    raw = dsp.timing(ym, 4)
    mod = cl.classify(raw, symmetry)
    # align BEFORE ddsync: the coarse carrier leaves only micro-Hz residual, so removing
    # the bulk phase first kills the loop transient that would corrupt the first symbols
    syms = _fine(raw, mod)
    return alpha, ym, raw, mod, syms, cl.quality(syms, mod)


def _rescue(eq_fn, z: np.ndarray, seed_mod: str, symmetry: int) -> tuple:
    """Equalize with a seed constellation, then let the OPENED eye pick the mod:
    heavy ISI corrupts the ring test and even the 2/8 symmetry vote, so re-classify
    with the estimated symmetry AND the neutral order-4 gate, keep the best lock."""
```

```
175 179     z1 = eq_fn(z, seed_mod)
176 180     cands = []
177 181     for m in {cl.classify(z1, symmetry), cl.classify(z1, 4)}:   # symmetry too, per the docstrin
    ↳ ↳       g
178 182         s = _fine(z1 if m == seed_mod else eq_fn(z, m), m)
179 183         cands.append((cl.quality(s, m), s, m))
180 184     best = max(cands, key=lambda c: c[0].lock)
181 185     # Occam de-fold for the PSK point-doubling: CMA is modulus-only, so a bpsk signal under a
182 186     # multipath echo ALSO fits a qpsk ring at a marginally higher lock (phantom 2->4 points).
183 187     # A bpsk candidate that clears the accept floor cannot be an over-fit of qpsk, so prefer it.
184 188     # Only fires when symmetry==2 put bpsk in the set; a genuine qpsk reads symmetry 4 -> the
185 189     # candidate set is the qpsk singleton -> this branch is unreachable and it stays untouched.
186 190     lo = [c for c in cands if c[2] == "bpsk" and c[0].lock >= _EQ_FLOOR]
187 191     if lo and best[2] == "qpsk":
188 192         return lo[0]
189 193     return best
190 194
191 195
192 196 def analyze(x: np.ndarray, meta: Meta, diff: bool = False,
193 197             burst: int | None = None) -> Result:
194 198     """Run the blind chain. `diff` demaps phase transitions (D-BPSK/D-QPSK);
195 199     `burst` selects one detected burst (default: the most energetic)."""
196 200     if x.size:
197 201         # pathological UNIFORM scale must be fixed BEFORE analytic(): its magnitude sanitizer
198 202         # zeroes samples above 1e150 sample-WISE, so a whole capture near/over that ceiling was
199 203         # wiped wholesale (garbage mods, lock=nan at 1e154) before the rms-normalize below could
200 204         # ever help. The 99th percentile is robust to spike outliers (a single corrupt 1e300
201 205         # sample leaves it at signal scale -> untouched here, still zeroed sample-wise later);
202 206         # normal captures sit far inside (1e-100, 1e100) -> untouched -> sweep byte-identical.
203 207         scale = float(np.percentile(np.abs(x[::16]), 99))  # strided: scale detection needs the
204 208         #             BULK magnitude, not every sample -- 16x cheaper on a large direct capture
205 209         if np.isfinite(scale) and scale > 0 and not (1e-100 < scale < 1e100):
206 210             x = x / scale
207 211     x = dsp.analytic(x)
208 212     x = x - x.mean()  # DC block: LO leakage otherwise corrupts every estimator
209 213     if x.size < 64:   # empty / trivially short: nothing to estimate, and it crashes downstream
210 214         raise ValueError(f"신호가 너무 짧습니다 ({x.size} 샘플)")
211 215     rms = float(np.sqrt(np.mean(np.abs(x) ** 2)))
212 216     if rms and (rms < 1e-6 or rms > 1e6):  # only PATHOLOGICAL scale: normalize so the scale-var
    ↳ ↳       iant
213 217         x = x / rms                       # spots (fsk_gate's CV epsilon, est_carrier's x**p ov
    ↳ ↳           er/
214 218     #                                       underflow) stay invariant. Normal captures rms~O(1
    ↳ ↳       ) ->
215 219     #                                       untouched -> the acceptance sweep is byte-identical
    ↳ ↳       .
216 220     bursts = dsp.find_bursts(x, meta.fs)
217 221     if burst is None or not 0 <= burst < len(bursts):
218 222         burst = int(np.argmax([np.sum(np.abs(x[s:e]) ** 2) for s, e in bursts]))
219 223     s, e = bursts[burst]
220 224     xb = x[s:e] if e - s >= 64 else x
221 225
226 +     # chirp gate: an FMCW/CSS sweep trips fsk_gate (bimodal IF) and would force-demodulate
227 +     # into confident garbage FSK. sweeps_band keeps real FSK (0/222 pass) out of this branch,
228 +     # so only a genuine band-sweep lands here -- characterized, never constellation-demodulated.
229 +     # The gates read a band-ISOLATED copy: mix to the spectral centroid, low-pass to the
230 +     # 99%-power band, and decimate so the signal fills ~28% of Nyquist. An off-centre
231 +     # narrowband sweep in a wide record otherwise dilutes the IF statistics with
232 +     # out-of-band noise and leaks through as garbage FSK.
233 +     pw = np.abs(np.fft.fft(xb)) ** 2                # sweep centre = spectral centroid
```

```
234 +     fr = np.fft.fftfreq(xb.size, 1 / meta.fs)       # (est_carrier means nothing on a mover)
235 +     fcg = float(np.sum(fr * pw) / (np.sum(pw) + 1e-30))
236 +     xg = dsp.mix(xb, meta.fs, fcg)
237 +     bwg = _bw99(xg, meta.fs)
238 +     want = max(1.25 * bwg, 1e-4 * meta.fs)
239 +     d = int(np.clip(0.28 * meta.fs / want, 1, max(1, xg.size // 256)))
240 +     fsg = meta.fs / d
241 +     xg = np.convolve(xg, firwin(129, max(min(0.9 * fsg / 2, bwg), 1e-4 * meta.fs)
242 +                                 / (meta.fs / 2)), "same")
243 +     if d > 1:
244 +         xg = resample_poly(xg, 1, d)
245 +     if is_chirp(xg, fsg) and sweeps_band(xg, fsg):
246 +         info = analyze_chirp(xg, fsg)
247 +         return Result(meta, "chirp", (s, e), fcg, info["rs"] or 0.0,
248 +                       "lora" if info["sf"] else "chirp", 0.0, np.zeros(0, complex),
249 +                       np.zeros(0, np.uint8), x, burst_x=xb,
250 +                       bursts=bursts, burst_idx=burst, chirp=info)
231 251
232 252     if fsk_gate(xb, meta.fs):
233 253         r = analyze_fsk(xb, meta.fs)
234 254         sym = np.asarray(r["symbols"], dtype=np.complex128)
235 255         return Result(meta, "fsk", (s, e), r["fc"], r["baud"], r["mod"], r["lock"],
236 256                       sym, r["bits"], xb, burst_x=xb, h=r["h"],
237 257                       bursts=bursts, burst_idx=burst)
238 258
239 259     fc0, symmetry, _ = dsp.est_carrier(xb, meta.fs, blocks=_blocks(xb.size))
240 260     fc = dsp.resolve_alias(xb, meta.fs, fc0, symmetry)
241 261     amb = abs(symmetry * fc) > 0.4 * meta.fs
242 262     xd = dsp.mix(xb, meta.fs, fc)
243 263
244 264     baud, conf = dsp.est_baud(xd, meta.fs)
245 265     fell = False
246 266     alpha, ym, raw, mod, syms, q = _demod(xd, meta.fs, baud, symmetry)
247 267     bw = dsp.occupied_bw(xd, meta.fs)
248 268     if bw and not (0.72 * bw <= baud <= 1.05 * bw):
249 269         # the global |x|^2 peak disagrees with the occupied band: below alpha~0.1
250 270         # (and under harsh channels) data junk outruns the true line. Demod the
251 271         # in-band hypotheses too and let lock decide, with a clear margin.
252 272         for lo in (0.80, 0.67):
253 273             b2, c2 = dsp.est_baud(xd, meta.fs, lo=lo * bw, hi=1.02 * bw)
254 274             if abs(b2 - baud) <= 0.01 * b2:
255 275                 continue
256 276             t = _demod(xd, meta.fs, b2, symmetry)
257 277             if t[5].lock > q.lock + _BAUD_GAIN:
258 278                 alpha, ym, raw, mod, syms, q = t
259 279                 baud, conf, fell = b2, c2, True
260 280
261 281     if q.lock < _SHORT_LOCK:
262 282         # short / low-SNR burst rescue: the default 4-block |x|^2 line splits the record
263 283         # too finely and locks a junk peak. Fewer, longer blocks give a stronger line (the
264 284         # carrier _blocks logic, applied to baud). Run both and keep only a clearly-better
265 285         # lock -- so a long, already-locked signal (CORE) never enters here and is untouched.
266 286         for nb in (2, 1):
267 287             b2, c2 = dsp.est_baud(xd, meta.fs, blocks=nb)
268 288             if abs(b2 - baud) <= 0.01 * b2:
269 289                 continue
270 290             t = _demod(xd, meta.fs, b2, symmetry)
271 291             if t[5].lock > q.lock + _BAUD_GAIN:
272 292                 alpha, ym, raw, mod, syms, q = t
273 293                 baud, conf, fell = b2, c2, True
```

```
274 294
275 295     eq_applied, eq_mode, pre = False, None, None
276 296     if q.lock < _EQ_LOCK:  # ISI (multipath) is what a phase-only loop cannot fix
277 297         # CMA is modulus-only, so it opens the eye without knowing the constellation.
278 298         qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
279 299         mode = "sym"
280 300         if qe.lock < max(_EQ_FLOOR, q.lock + _EQ_GAIN):
281 301             # symbol-spaced FIR could not invert the channel; retry T/2-spaced on
282 302             # the clock-tracked 2 sps stream (unaliased spectrum, longer inverse)
283 303             q2, s2, m2 = _rescue(lambda z, m: equalize_fse(z, m),
284 304                                  dsp.timing(ym, 4, out=2), mod, symmetry)
285 305             if q2.lock > qe.lock:
286 306                 qe, se, me, mode = q2, s2, m2, "fse"
287 307         floor = _EQ_QAM_FLOOR if me in ("32qam", "64qam") else _EQ_FLOOR
288 308         if qe.lock > q.lock + _EQ_GAIN and qe.lock >= floor:
289 309             syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
290 310     elif mod in ("16qam", "32qam", "64qam"):
291 311         # confident square-QAM at high lock: a benign post-echo can fold a LOWER-order signal
292 312         # onto a QAM lattice with a clean-looking eye, so the low-lock rescue above never fires.
293 313         # Equalize once and accept ONLY if a strictly lower order emerges -- verified never to
294 314         # demote a genuine QAM (0/36 across 16/32/64qam x snr x seeds), so real QAM is untouched
    ↳ ↳           .
295 315         qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
296 316         mode = "sym"
297 317         if mod_order(me) < mod_order(mod) and qe.lock < _EQ_FLOOR:
298 318             # a lower order emerged but the symbol-spaced eye stayed shut (echo > ~2 symbols);
299 319             # retry T/2 fractionally-spaced. Only runs once a de-fold is INDICATED, so a genuine
300 320             # QAM (no lower order) never pays for the FSE.
301 321             qe, se, me = _rescue(lambda z, m: equalize_fse(z, m),
302 322                                  dsp.timing(ym, 4, out=2), mod, symmetry)
303 323             mode = "fse"
304 324         if mod_order(me) < mod_order(mod) and qe.lock >= _EQ_FLOOR:
305 325             syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
306 326
307 327     if q.lock < _SYNC_LOCK:
308 328         # packetized-burst rescue -- LAST, after the equalizer: a repeated preamble pins the
309 329         # carrier/baud/timing from only a few symbols, where the blind M-th-power estimator (nee
    ↳ ↳           ding
310 330         # many symbols to average) fails. Runs only if the eq rescue above did NOT lift the lock
    ↳ ↳           , so
311 331         # a multipath signal (which the eq handles, and whose ISI can fake a short period) never
312 332         # reaches here. Detect the preamble, mix by its CFO, demod the DATA past it, keep-best -
    ↳ ↳           - a
313 333         # random / no-preamble signal yields no plateau or a CFO that does not improve lock an
    ↳ ↳           d is
314 334         # dropped (the sweep stays byte-identical). The preamble period P = L*sps supplies the b
    ↳ ↳           aud
315 335         # (fs*L/P per block length L), the phase slope gives the carrier (+-fs/P resolves the L-
    ↳ ↳           fold
316 336         # alias); every PSK/QAM symmetry is tried -- the right (carrier, baud, sym) locks clean.
317 337         ps = find_preamble(xb, meta.fs, baud_hint=baud)
318 338         if ps is not None:
319 339             # The Schmidl-Cox CFO is ambiguous modulo fs/P = baud/L: an M-PSK carrier off by bau
    ↳ ↳               d/L
320 340             # rotates a whole constellation position per symbol, so the eye still looks locked b
    ↳ ↳               ut
321 341             # the bits shift -> a confident WRONG decode if the wrong alias is trusted. The coar
    ↳ ↳               se
322 342             # M-th-power fc lands within ~1 alias step of the truth even on these short bursts
    ↳ ↳               , so
```

```
323 343            # CENTRE the alias scan on it (k0) and sweep +-3 steps -- this reaches the correct a
    ↳ ↳              lias
324 344            # for any offset (not just |fc|<baud/2); keep-best over them picks the cleanest eye.
325 345            step = meta.fs / ps.period
326 346            k0 = round((fc - ps.cfo_hz) / step)
327 347            cands = []
328 348            for b2 in sorted({round(meta.fs * L / ps.period) for L in (3, 4, 6, 8)}):
329 349                for k in range(k0 - 3, k0 + 4):
330 350                    cfo = ps.cfo_hz + k * step
331 351                    data = dsp.mix(xb, meta.fs, cfo)[ps.end:]
332 352                    if data.size < 256:
333 353                        continue
334 354                    for sm in (2, 4, 8):
335 355                        t = _demod(data, meta.fs, float(b2), sm)
336 356                        cands.append((t[5].lock, cfo, float(b2), sm, t))
337 357            cands.sort(key=lambda c: -c[0])
338 358            # Adjacent aliases both look like clean M-PSK, so lock alone can pick the WRONG on
    ↳ ↳              e by a
339 359            # hair. Require the winner to beat the best DIFFERENT-carrier candidate by a clear g
    ↳ ↳              ap;
340 360            # otherwise the alias is genuinely ambiguous -> leave the honest low-lock result rat
    ↳ ↳              her
341 361            # than emit a confident wrong decode. (Same-carrier baud/sym variants do not compete
    ↳ ↳              .)
342 362            # PRECISION GATE: the alias ambiguity (carrier off by baud/L) is blind-ambiguous -
    ↳ ↳              - a
343 363            # rotated alias still locks AS HIGH AS the truth (both are clean M-PSK), so lock alo
    ↳ ↳              ne
344 364            # cannot separate them (a rotated 8psk alias was measured locking 78 with ber 0.38)
    ↳ ↳              . Two
345 365            # conjunctive conditions favour precision over recall: (1) a strong absolute lock, a
    ↳ ↳              nd
346 366            # (2) the winner sits within _SYNC_ADIST alias steps of the coarse est_carrier fc -
    ↳ ↳              - a
347 367            # baud/L rotation is >=1 step off the true carrier, so a far-from-anchor winner is a
348 368            # rotation. Genuine recoveries have a near-anchor carrier (adist<=0.5, lock 88+); o
    ↳ ↳              n the
349 369            # hardest bursts the anchor itself is unreliable -> both conditions fail -> refuse.
350 370            if cands and cands[0][0] >= _SYNC_ACCEPT and cands[0][0] > q.lock + _EQ_GAIN:
351 371                top = cands[0]
352 372                other = next((c for c in cands if abs(c[1] - top[1]) > step / 2), None)
353 373                anchored = abs(top[1] - fc) <= _SYNC_ADIST * step
354 374                if anchored and (other is None or top[0] - other[0] >= _ALIAS_MARGIN):
355 375                    alpha, ym, raw, mod, syms, q = top[4]
356 376                    fc, baud, symmetry = top[1], top[2], top[3]
357 377                    eq_applied, eq_mode, pre = False, None, ps
358 378                    # diagnostics must describe the ACCEPTED decode, not the abandoned blind
359 379                    # attempt: the carrier came from the preamble (not resolve_alias -> fc0=fc
360 380                    # keeps alias_resolved False), the baud from the preamble period (no
361 381                    # spectral line -> conf 0, no fallback), and ambiguity follows the new fc.
362 382                    fc0, conf, fell = fc, 0.0, False
363 383                    amb = abs(symmetry * fc) > 0.4 * meta.fs
364 384
365 385        bits = demap_diff_bits(syms, _DIFF_OF[mod]) if diff and mod in _DIFF_OF \
366 386            else demap_bits(syms, mod)
367 387        return Result(meta, "linear", (s, e), fc, baud, mod, q.lock, syms, bits, ym,
368 388                      burst_x=xb, symmetry=symmetry, carrier_ambiguous=amb, baud_conf=conf,
369 389                      rolloff=alpha, evm=q.evm, mer_db=q.mer_db, occupied=q.occupied,
370 390                      snr_est=cl.snr_m2m4(syms, mod), eq_applied=eq_applied, eq_mode=eq_mode,
371 391                      alias_resolved=abs(fc - fc0) > 1.0, baud_fallback=fell,
```

```
372 392                 bursts=bursts, burst_idx=burst, preamble=pre)
373 393
374 394
375 395 def analyze_file(path: str, fs: float | None = None, fmt: str | None = None,
376 396                 dtype: str | None = None, endian: str | None = None,
377 397                 bitrev: bool | None = None, diff: bool = False,
378 398                 burst: int | None = None, rf: float | None = None) -> Result:
379 399     """Analyze a file; explicit args override SigMF/filename tokens."""
380 400     from .sigio import parse_sigmf
381 401     m = parse_sigmf(path) or parse_name(path)
382 402     meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
383 403                 endian or m.endian, m.bitrev if bitrev is None else bitrev,
384 404                 rf_center=rf if rf is not None else m.rf_center)
385 405     x, meta = read(path, meta)
386 406     return analyze(x, meta, diff, burst)
387 407
388 408
409 + # --- web payload ------------------------------------------------------------
390 410
411 + def analyze_web(x: np.ndarray, meta: Meta, *, burst: int | None = None) -> dict:
412 +     """Payload for POST /api/analyze: the analyze() result as JSON, plus -- on the first
413 +     load of a multi-burst capture -- a whole-record waterfall so the UI can draw the burst
414 +     map. Burst re-selection posts again with ?burst=N and skips the overview (the UI keeps
415 +     the one it already has)."""
416 +     r = analyze(x, meta, burst=burst)
417 +     out = r.to_json()
418 +     if burst is None and len(r.bursts) > 1:
419 +         xa = dsp.analytic(x)
420 +         xa = xa - xa.mean()
421 +         out["overview"] = {"n": int(xa.size), "fs": meta.fs,
422 +                            "waterfall": waterfall(xa, meta.fs)}
423 +     return out
```

signus/cli.py +37 −81 · **코드북 39쪽** · 새로 쓸 줄 1, 10, 42-53, 57, 69-89, 114

```
1 + """CLI: analyze FILE / serve. 합성·채점(gen/dataset/sweep)은 개발기 전용 lab.py 가
2 2 단다 — 격리망 장비에는 그 파일이 없어 명령 자체가 없다."""
3 3
4 4 import argparse
5 5 import importlib.util
6 6 import re
7 7 import sys
8 8 from zlib import crc32
9 9
10 + from .pipeline import analyze_file
11 11 from .sigio import sidecar_read
12 12
13 13 _DTYPE_CHOICES = ("i8", "u8", "i16", "u16", "f32", "f64")
14 14
15 15
16 16 # --- 손으로 옮기는 한 줄 ----------------------------------------------------
17 17 # 실신호 장비는 인터넷도 클립보드도 없다. 결과는 사람이 화면을 보고 받아쳐서 나온다. 그래서
18 18 # 이 블록은 반드시 여기(필사 대상)에 있어야 한다 -- tools/ 에 두면 그 도구부터 필사해야 하는
19 19 # 본말전도가 된다. 줄 끝의 검출 코드가 "옮기다 한 글자 틀림"을 잡는다.
20 20
21 21 _ALPHA = "0123456789abcdefghjkmnpqrstvwxyz"  # 0/o, 1/l/i 처럼 화면에서 헷갈리는 글자를 뺀 32자
22 22 _BRIEF_FLAGS = [("eq", lambda d: d["eq"]["applied"]),
23 23                 ("al", lambda d: d["detected"]["alias_resolved"]),
24 24                 ("fb", lambda d: d["detected"]["baud_fallback"]),
25 25                 ("amb", lambda d: d["detected"]["carrier_ambiguous"]),
26 26                 ("pre", lambda d: "preamble" in d["detected"])]
```

```
 27  27
 28  28
 29  29  def check_code(text: str) -> str:
 30  30      """받아친 줄의 오타 검출 코드 (4글자 = 20비트). 공백은 '한 칸으로 줄이되 없애지는'
 31  31      않는다 -- 전부 지우면 'fs1000000 16qam' 과 'fs10000001 6qam'(샘플레이트 10배!) 이 같은
 32  32      코드가 되어, 값이 바뀌는 오타 32종이 조용히 통과했다. 대소문자와 줄 간격은 계속 무시한다."""
 33  33      body = re.sub(r"\s+", " ", re.sub(r"#[0-9a-z]{4}\b", "", text.lower())).strip()
 34  34      n = crc32(body.encode())
 35  35      return "".join(_ALPHA[(n >> (5 * i)) & 31] for i in (3, 2, 1, 0))
 36  36
 37  37
 38  38  def brief(doc: dict, mode: str) -> str:
 39  39      """손으로 옮기는 한 줄 + 검출 코드. Result.to_json() 딕셔너리에서 만든다 -- 객체
 40  40      속성을 직접 포맷하면 to_json 이 이미 한 반올림과 두 번 겹쳐 장비와 맥이 갈린다."""
 41  41      head = f"sig2 {mode} fs{doc['fs']:.0f} {doc['fmt']}-{doc['dtype']}"
     42 +    d, q = doc["detected"], doc["quality"]
     43 +    p = [head, d["mod"], f"fc{d['fc']:.0f}", f"bd{d['baud']:.0f}", f"lk{q['lock']:.0f}"]
     44 +    if q["mer_db"] is not None:
     45 +        p.append(f"mer{q['mer_db']:.1f}")
     46 +    if d.get("h") is not None:                    # FSK 는 롤오프 대신 변조지수
     47 +        p.append(f"h{d['h']:.2f}")
     48 +    elif d["rolloff"] is not None:
     49 +        p.append(f"rl{d['rolloff']:.2f}")
     50 +    if len(doc["bursts"]) > 1:
     51 +        p.append(f"b{doc['burst_idx'] + 1}/{len(doc['bursts'])}")
     52 +    p += [f for f, get in _BRIEF_FLAGS if get(doc)]
     53 +    body = " ".join(p)
 64  54      return f"{body} #{check_code(body)}"
 65  55
 66  56
     57 + # --- analyze / serve ------------------------------------------------------------
 68  58
 69  59  def _analyze(args: argparse.Namespace) -> int:
 70  60      r = analyze_file(args.file, args.fs, args.fmt, args.dtype,
 71  61                       "be" if args.be else None, args.bitrev or None, args.diff,
 72  62                       None if args.burst is None else args.burst - 1, rf=args.rf)
 73  63      j = r.to_json(views=False)          # 한 줄 요약도 여기서 만든다 (반올림이 한 번만 걸리게)
 74  64      d = j["detected"]
 75  65      truth = (sidecar_read(args.file) or {}).get("truth")  # display only, never detection
 76  66      if len(r.bursts) > 1:
 77  67          print(f"시간상 버스트 {len(r.bursts)}개 — 그중 {r.burst_idx + 1}번을 분석"
 78  68                " (--burst N 으로 선택)")
     69 +    c = d.get("chirp")
     70 +    if c:                                 # 처프/CSS: 성상도 제원이 없다 — 특성으로 보고
     71 +        print("변조       " + (f"LoRa 추정 SF{c['sf']}" if c["sf"] else "처프(FMCW/CSS)"))
     72 +        print(f"중심주파수 {d['fc']:.1f} Hz")
     73 +        if d["rf_hz"] is not None:
     74 +            print(f"실제 RF   {d['rf_hz'] / 1e6:.6f} MHz")
     75 +        print(f"점유대역폭 {c['bw']:.0f} Hz   방향 {'상승' if c['up'] else '하강'}"
     76 +              f"   처프율 {c['mu']:.3g} Hz/s")
     77 +        if c["sf"]:
     78 +            print(f"심볼레이트 {c['rs']:.1f} Hz   심볼시간 {c['tsym'] * 1e3:.2f} ms")
     79 +    else:
     80 +        print(f"변조       {d['mod']}" + (f"   (정답 {truth['mod']})" if truth else ""))
     81 +        print(f"중심주파수 {d['fc']:.1f} Hz" + (f"   (정답 {truth['fc']:.1f})" if truth else "")
      ↳ +          )
     82 +        if d["rf_hz"] is not None:
     83 +            print(f"실제 RF   {d['rf_hz'] / 1e6:.6f} MHz")
     84 +        print(f"baud       {d['baud']:.1f} Hz"
     85 +              + (f"   (정답 {truth['baud']:.1f})" if truth else ""))
```

```
    +       fsk = r.family == "fsk"
    +       tail = f"변조지수 h {d['h']:.2f}" if fsk else f"롤오프    {d['rolloff']:.2f}"
    +       mer = "" if fsk else f"   MER {r.mer_db:.1f} dB"
    +       print(f"{tail}   lock {r.lock:.1f}{mer}")
 88 90      if r.eq_applied:
 89 91          print("다중경로 보정(등화기) 적용"
 90 92                + (" (T/2 분수간격)" if r.eq_mode == "fse" else " (심볼간격)"))
 91 93      if r.alias_resolved:
 92 94          print("중심주파수 접힘(앨리어스) 보정: 후보 중 스펙트럼 무게중심에 맞는 것을 선택")
 93 95      if r.baud_fallback:
 94 96          print("baud 폴백: 스펙트럼에서 baud 선을 못 찾아 점유대역폭으로 추정 — baud 신뢰 낮음")
 95 97      if r.carrier_ambiguous:
 96 98          print("경고: 중심주파수가 접혀 있을 수 있음 — fs 에 비해 너무 높다"
 97 99                " (fc 를 그대로 믿지 말 것)")
 98 100     if args.save_iq:
 99 101         r.save_iq(args.save_iq)
100 102     if args.save_symbols:
101 103         r.save_symbols(args.save_symbols)
102 104     if args.save_bits:
103 105         r.save_bits(args.save_bits, args.packed)
104 106     if args.report:
105 107         r.save_report(args.report)
106 108     if args.brief:                          # 사람용 출력을 대체하지 않고 맨 끝에 한 줄 더 --
107 109         print(brief(j, "an"))   # 조기 return 이면 위 저장들이 조용히 무시된다
108 110     return 0
109 111
110 112
149 113 def _read_args(p: argparse.ArgumentParser) -> None:
    114 +     """Read-side sample-format overrides for analyze (sidecar/filename win)."""
151 115     p.add_argument("--fs", type=float)
152 116     p.add_argument("--fmt", choices=("iq", "real"))
153 117     p.add_argument("--dtype", choices=_DTYPE_CHOICES)
154 118     p.add_argument("--be", action="store_true", help="big-endian 샘플")
155 119     p.add_argument("--bitrev", action="store_true", help="바이트 내 비트 역순")
156 120     p.add_argument("--rf", type=float, help="RF 중심주파수 (Hz) — 실제 주파수로 보고")
157 121     p.add_argument("--brief", action="store_true",
158 122                    help="손으로 옮길 한 줄 + 오타 검출 코드를 끝에 덧붙임")
159 123
160 124
161 125 def main(argv: list[str] | None = None) -> int:
162 126     ap = argparse.ArgumentParser(prog="signus")
163 127     sub = ap.add_subparsers(dest="cmd", required=True)
164 128
165 129     a = sub.add_parser("analyze", help="블라인드 복조 + 제원 탐지")
166 130     a.add_argument("file")
167 131     _read_args(a)
168 132     a.add_argument("--save-iq")
169 133     a.add_argument("--save-symbols")
170 134     a.add_argument("--save-bits")
171 135     a.add_argument("--packed", action="store_true")
172 136     a.add_argument("--report")
173 137     a.add_argument("--diff", action="store_true",
174 138                    help="차동 디맵(D-BPSK/D-QPSK): 회전 모호성 없이 비트 복원")
175 139     a.add_argument("--burst", type=int, help="분석할 버스트 번호 (1부터; 기본 최강 버스트)")
176 140
183 141     s = sub.add_parser("serve", help="웹 UI 서버")
184 142     s.add_argument("--host", default="127.0.0.1")
185 143     s.add_argument("--port", type=int, default=8000)
186 144
187 145     # 합성 생성·채점(gen/dataset/sweep)은 개발기 전용이다. 격리망 장비에는 lab.py 를
```

```
188 146     # 옮기지 않으므로 그 장비에서는 이 명령들이 아예 존재하지 않는다. try/except 로 삼키면
189 147     # 개발기에서 lab 내부의 진짜 임포트 오류까지 "명령 없음"으로 둔갑하니 파일 유무만 본다.
190 148     if importlib.util.find_spec("signus.lab"):
191 149         from . import lab
192 150         lab.add_commands(sub)
193 151
194 152     args = ap.parse_args(argv)
195 153     if args.cmd == "analyze":
196 154         return _analyze(args)
199 155     if args.cmd == "serve":
200 156         from .server import run
201 157         run(args.host, args.port)
202 158         return 0
203 159     return args.run(args)           # lab 이 단 명령 (gen / dataset / sweep)
204 160
205 161
206 162 if __name__ == "__main__":
207 163     sys.exit(main())
```

signus/server.py +3 −4 · **코드북 42쪽** · 새로 쓸 줄 9, 45, 66

```
1   1  """Stdlib-only web server: static frontend + POST /api/analyze (raw body,
2   2  filename + optional fs/fmt/dtype overrides in the query string — no multipart)."""
3   3
4   4  import json
5   5  from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
6   6  from pathlib import Path
7   7  from urllib.parse import parse_qs, urlparse
8   8
    9 + from .pipeline import analyze_web
10 10  from .sigio import Meta, decode, parse_name
11 11
12 12  _WEB = Path(__file__).resolve().parent / "web"  # inside the package so wheels/sdists ship it
13 13  _MIME = {".html": "text/html", ".css": "text/css", ".js": "text/javascript",
14 14       ".json": "application/json", ".png": "image/png", ".svg": "image/svg+xml"}
15 15  _MAX_BODY = 256 * 1024 * 1024
16 16
17 17
18 18  class Handler(BaseHTTPRequestHandler):
19 19      def log_message(self, *_: object) -> None:
20 20          pass
21 21
22 22      def _json(self, code: int, obj: dict) -> None:
23 23          body = json.dumps(obj).encode()
24 24          self.send_response(code)
25 25          self.send_header("Content-Type", "application/json")
26 26          self.send_header("Content-Length", str(len(body)))
27 27          self.end_headers()
28 28          self.wfile.write(body)
29 29
30 30      def _meta(self, q: dict) -> Meta:
31 31          """Build Meta from query params with filename fallback; raise on unknown."""
32 32          name = q.get("name", [""])[0]
33 33          m = parse_name(name)
34 34          meta = Meta(float(q["fs"][0]) if "fs" in q else m.fs,
35 35                      q.get("fmt", [m.fmt])[0], q.get("dtype", [m.dtype])[0],
36 36                      q.get("endian", [m.endian])[0],
37 37                      q.get("bitrev", ["1" if m.bitrev else "0"])[0] == "1",
38 38                      rf_center=float(q["rf"][0]) if "rf" in q else m.rf_center)
39 39          if not meta.ok():
40 40              raise ValueError(f"샘플레이트/포맷을 알 수 없습니다: {name}")
```

```
41 41          return meta
42 42
43 43      def do_POST(self) -> None:  # noqa: N802 (BaseHTTPRequestHandler API)
44 44          url = urlparse(self.path)
   45 +        if url.path != "/api/analyze":
46 46              self._json(404, {"error": "not found"})
47 47              return
48 48          q = parse_qs(url.query)
49 49          n = int(self.headers.get("Content-Length") or 0)
50 50          if n <= 0:
51 51              self._json(411, {"error": "Content-Length 필요"})
52 52              return
53 53          if n > _MAX_BODY:
54 54              self._json(413, {"error": "파일이 너무 큽니다 (최대 256MB)"})
55 55              return
56 56          try:
57 57              meta = self._meta(q)
58 58              burst = int(q["burst"][0]) if "burst" in q else None
59 59              x = decode(self.rfile.read(n), meta)
60 60              if x.size < 256:
61 61                  raise ValueError("신호가 너무 짧습니다")
62 62          except ValueError as exc:
63 63              self._json(400, {"error": str(exc)})
64 64              return
65 65          try:
   66 +            out = analyze_web(x, meta, burst=burst)
67 67              self._json(200, out)
68 68          except Exception as exc:  # surface DSP failures to the UI
69 69              self._json(500, {"error": f"분석 실패: {exc}"})
70 70
71 71      def do_GET(self) -> None:  # noqa: N802
72 72          rel = urlparse(self.path).path.lstrip("/") or "index.html"
73 73          target = (_WEB / rel).resolve()
74 74          if not (target.is_file() and target.is_relative_to(_WEB.resolve())):
75 75              self._json(404, {"error": "not found"})
76 76              return
77 77          body = target.read_bytes()
78 78          self.send_response(200)
79 79          self.send_header("Content-Type", _MIME.get(target.suffix, "application/octet-stream"))
80 80          self.send_header("Content-Length", str(len(body)))
81 81          self.end_headers()
82 82          self.wfile.write(body)
83 83
84 84
85 85  def run(host: str = "127.0.0.1", port: int = 8000) -> None:
86 86      srv = ThreadingHTTPServer((host, port), Handler)
87 87      print(f"signus UI → http://{host}:{srv.server_address[1]}")
88 88      srv.serve_forever()
```

signus/web/index.html +1 −21 · 코드북 44쪽 · 새로 쓸 줄 114

```
1 1  <!doctype html>
2 2  <html lang="ko">
3 3  <head>
4 4  <meta charset="utf-8">
5 5  <meta name="viewport" content="width=device-width, initial-scale=1">
6 6  <title>Signus — 신호 복조 분석기</title>
7 7  <link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='
↳ ↳    0 0 16 16'%3E%3Crect width='16' height='16' rx='5' fill='%233182F6'/%3E%3C/svg%3E">
8 8  <link rel="stylesheet" href="/style.css">
9 9  <script defer src="/app.js"></script>
```

```
</head>
<body>
<header class="topbar">
  <div class="brand">
    <span class="logo" aria-hidden="true"></span>
    <div>
      <h1>Signus</h1>
      <p class="tagline">블라인드 PSK/QAM/FSK 신호 복조 분석기</p>
    </div>
  </div>
</header>

<main class="wrap">
  <section id="dropCard" class="card drop" tabindex="0" role="button"
           aria-label="신호 파일 선택 또는 드래그">
    <input id="fileInput" type="file" multiple hidden>
    <div class="drop-icon" aria-hidden="true"></div>
    <p class="drop-title">신호 파일을 여기에 놓으세요</p>
    <p class="drop-sub">여러 개를 놓으면 일괄 분석 · <code>.json</code> 정답 · <code>.sigmf-meta
      </code> 지원</p>
  </section>

  <section id="metaForm" class="card form hidden">
    <p class="form-title">파일 이름에서 정보를 찾지 못했어요. 직접 입력해주세요.</p>
    <div class="form-row">
      <label>샘플레이트 (Hz)
        <input id="mFs" type="number" min="1" step="any" placeholder="예: 1000000"></label>
      <label>포맷
        <select id="mFmt">
          <option value="">선택</option><option value="iq">IQ</option><option value="real">Real<
            /option>
        </select></label>
      <label>데이터 타입
        <select id="mDtype">
          <option value="i16">i16 (16t)</option><option value="i8">i8 (8t)</option>
          <option value="u8">u8 (8o)</option><option value="u16">u16 (16o)</option>
          <option value="f32">f32</option><option value="f64">f64</option>
        </select></label>
    </div>
    <button id="metaGo" class="btn">분석 시작</button>
  </section>

  <section id="loading" class="card loading hidden" aria-live="polite">
    <div class="spinner" aria-hidden="true"></div>
    <p id="loadMsg">신호를 분석하고 있어요…</p>
  </section>

  <section id="errorCard" class="card error hidden" role="alert">
    <div class="err-mark" aria-hidden="true">!</div>
    <div>
      <p class="err-title">분석에 실패했어요</p>
      <p id="errMsg" class="err-msg"></p>
    </div>
  </section>

  <!-- Batch results table -->
  <section id="batchCard" class="card hidden">
    <div class="card-head">
      <h2 class="card-title">일괄 분석 결과</h2>
      <button id="dlCsv" class="btn ghost small">CSV 내보내기</button>
```

```
68 68     </div>
69 69     <div class="table-wrap">
70 70       <table class="batch">
71 71         <thead><tr><th>파일</th><th>변조</th><th>중심주파수</th><th>심볼레이트</th><th>Lock</th>
   ↳ ↳           </tr></thead>
72 72         <tbody id="batchBody"></tbody>
73 73       </table>
74 74     </div>
75 75     <p class="hint">행을 클릭하면 상세 화면으로 이동합니다.</p>
76 76   </section>
77 77
98 78   <section id="results" class="results hidden">
99 79     <button id="backBatch" class="btn ghost small hidden">← 목록으로</button>
100 80
101 81    <div class="card summary">
102 82      <div class="summary-head">
103 83        <span id="statusPill" class="pill"></span>
104 84        <p id="fileName" class="file-name"></p>
105 85      </div>
106 86      <div class="summary-body">
107 87        <div class="gauge">
108 88          <svg viewBox="0 0 120 120" class="donut" aria-hidden="true">
109 89            <circle class="donut-track" cx="60" cy="60" r="52"></circle>
110 90            <circle id="donutArc" class="donut-arc" cx="60" cy="60" r="52"></circle>
111 91          </svg>
112 92          <div class="gauge-center"><span id="lockNum" class="gauge-num">0</span>
113 93            <span class="gauge-cap tip" title="별자리가 얼마나 조밀·정렬됐는지 (0~100). 60 이상
   ↳ ↳             이면 복조 신뢰">LOCK</span></div>
114 94        </div>
115 95        <div class="metrics">
116 96          <div class="metric"><span class="metric-label tip" title="변조 오차비 — 높을수록 깨
   ↳ ↳             끗 (dB)">MER</span>
117 97            <span id="merVal" class="metric-val">—</span></div>
118 98          <div class="metric"><span class="metric-label tip" title="오차 벡터 크기 — 낮을수록 좋
   ↳ ↳             음 (%)">EVM</span>
119 99            <span id="evmVal" class="metric-val">—</span></div>
120 100         <div class="metric"><span class="metric-label tip" title="추정 심볼 신호 대 잡음비 (dB
   ↳ ↳             )">SNR</span>
121 101           <span id="snrVal" class="metric-val">—</span></div>
122 102       </div>
123 103     </div>
124 104     <div id="flagRow" class="chips"></div>
125 105     <div id="burstRow" class="chips"></div>
126 106   </div>
127 107
128 108   <div id="burstMap" class="card hidden">
129 109     <h2 class="card-title">버스트 지도</h2>
130 110     <div class="wf-wrap">
131 111       <canvas id="burstFall" class="fall"></canvas>
132 112       <div id="burstBoxes" class="boxes"></div>
133 113     </div>
    114 +     <p class="hint">전체 녹음의 스펙트로그램 (가로=시간, 위=+주파수) · 위 번호와 아래 띠가 검
    ↳ +       출된 버스트. 구간을 클릭하면 그 구간을 분석합니다.</p>
135 115   </div>
136 116
137 117   <div class="card">
138 118     <h2 class="card-title">스펙트럼 · 워터폴</h2>
139 119     <canvas id="specCanvas" class="spec"></canvas>
140 120     <canvas id="fallCanvas" class="fall"></canvas>
141 121   </div>
```

```
142 122
143 123     <div class="card const-card">
144 124       <div class="card-head">
145 125         <h2 class="card-title">성상도</h2>
146 126         <div class="play-row">
147 127           <button id="playBtn" class="btn ghost small" aria-label="재생/일시정지">⏸ 일시정지</bu
    ↳ ↳             tton>
148 128           <select id="speedSel" class="sel small" aria-label="재생 속도">
149 129             <option value="1">1x</option><option value="4" selected>4x</option>
150 130             <option value="16">16x</option>
151 131           </select>
152 132         </div>
153 133       </div>
154 134       <canvas id="constCanvas" class="const"></canvas>
155 135       <input id="scrub" class="scrub" type="range" min="0" max="1000" value="0"
156 136           aria-label="재생 위치">
157 137       <p id="playInfo" class="hint"></p>
158 138     </div>
159 139
160 140     <div id="paramGrid" class="param-grid"></div>
161 141
162 142     <div class="card downloads">
163 143       <h2 class="card-title">다운로드</h2>
164 144       <div class="dl-row">
165 145         <button id="copyBits" class="btn ghost">비트 복사</button>
166 146         <button id="dlBits" class="btn ghost">비트 .txt</button>
167 147         <button id="dlSyms" class="btn ghost">심볼 .csv</button>
168 148         <button id="dlReport" class="btn ghost">리포트 .json</button>
169 149       </div>
170 150     </div>
171 151   </section>
172 152 </main>
173 153 </body>
174 154 </html>
```

signus/web/style.css +5 −21 · **코드북 47쪽** · 새로 쓸 줄 194, 199-200, 205, 213

```
1  1  :root {
2  2    --bg: #f4f6f8; --card: #fff; --ink: #191f28; --sub: #6b7684;
3  3    --blue: #3182F6; --blue-weak: #e8f1fe; --line: #eef1f4;
4  4    --green: #12b886; --amber: #f59f00; --red: #fa5252;
5  5    --field: #fff; --hover: #f7fafd; --border: #dfe4ea;    /* theme-sensitive surfaces */
6  6    --ok-bg: #e6f7f1; --warn-bg: #fff4e0; --bad-bg: #ffecec;
7  7    --radius: 20px; --shadow: 0 6px 24px rgba(30, 42, 66, .07);
8  8  }
9  9  /* dark mode: follows the OS. The signal canvases are already dark, so they blend in. */
10 10 @media (prefers-color-scheme: dark) {
11 11   :root {
12 12     --bg: #0e131b; --card: #161d28; --ink: #e6ebf2; --sub: #94a1b2;
13 13     --blue-weak: #17273c; --line: #232c39; --field: #1b2330; --hover: #1d2632;
14 14     --border: #2b3543; --ok-bg: #102c24; --warn-bg: #302512; --bad-bg: #321b1e;
15 15     --shadow: 0 6px 24px rgba(0, 0, 0, .38);
16 16   }
17 17 }
18 18
19 19 * { box-sizing: border-box; }
20 20
21 21 body {
22 22   margin: 0; background: var(--bg); color: var(--ink);
23 23   font-family: -apple-system, 'Apple SD Gothic Neo', 'Segoe UI', 'Malgun Gothic', sans-serif;
24 24   line-height: 1.5; -webkit-font-smoothing: antialiased;
```

```
}

.wrap { max-width: 580px; margin: 0 auto; padding: 8px 20px 64px; }

.topbar { max-width: 580px; margin: 0 auto; padding: 28px 20px 12px; }
.brand { display: flex; align-items: center; gap: 12px; }
.logo {
  width: 40px; height: 40px; border-radius: 12px; flex: none;
  background: linear-gradient(135deg, var(--blue), #6aa8ff);
}
h1 { margin: 0; font-size: 22px; font-weight: 800; letter-spacing: -.4px; }
.tagline { margin: 2px 0 0; color: var(--sub); font-size: 13px; }

.card {
  background: var(--card); border-radius: var(--radius); padding: 20px;
  box-shadow: var(--shadow); margin-top: 16px; animation: rise .35s ease both;
}
.card-title { margin: 0 0 12px; font-size: 15px; font-weight: 700; }
.hidden { display: none !important; }

@keyframes rise { from { opacity: 0; transform: translateY(10px); } to { opacity: 1; transform
  : none; } }

/* Dropzone */
.drop {
  text-align: center; cursor: pointer; border: 2px dashed var(--border);
  transition: border-color .18s, background .18s, transform .18s;
}
.drop:hover, .drop:focus-visible { border-color: var(--blue); background: var(--hover); outline
  : none; }
.drop.drag { border-color: var(--blue); background: var(--blue-weak); transform: scale(1.01); }
.drop-icon {
  width: 52px; height: 52px; margin: 6px auto 14px; border-radius: 16px;
  background: var(--blue-weak); position: relative;
}
.drop-icon::after {
  content: ''; position: absolute; inset: 16px 15px; border: 3px solid var(--blue);
  border-radius: 3px; border-top: none; border-right: none;
  transform: rotate(45deg) translate(2px, -2px);
}
.drop-title { margin: 0; font-size: 16px; font-weight: 700; }
.drop-sub { margin: 6px 0 0; color: var(--sub); font-size: 13px; }
code { background: var(--line); padding: 1px 6px; border-radius: 6px; font-size: 12px; }

/* Meta form */
.form-title { margin: 0 0 12px; font-size: 14px; font-weight: 600; }
.form-row { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 12px; }
.form-row label { display: flex; flex-direction: column; gap: 6px; font-size: 12px; color: var(-
  -sub); }
input, select {
  font: inherit; padding: 10px 12px; border: 1px solid var(--border); border-radius: 12px;
  background: var(--field); color: var(--ink);
}
input:focus, select:focus { outline: 2px solid var(--blue); border-color: transparent; }

.btn {
  margin-top: 16px; width: 100%; padding: 13px; border: none; border-radius: 14px;
  background: var(--blue); color: #fff; font-size: 15px; font-weight: 700; cursor: pointer;
  transition: filter .15s, transform .1s;
}
```

```
.btn:hover { filter: brightness(1.05); }
.btn:active { transform: scale(.98); }
.btn.ghost { background: var(--blue-weak); color: var(--blue); margin-top: 0; }

/* Loading */
.loading { display: flex; align-items: center; gap: 14px; color: var(--sub); }
.spinner {
  width: 26px; height: 26px; border: 3px solid var(--line);
  border-top-color: var(--blue); border-radius: 50%; animation: spin .8s linear infinite;
}
@keyframes spin { to { transform: rotate(360deg); } }

/* Error */
.error { display: flex; gap: 14px; align-items: center; border-left: 4px solid var(--red); }
.err-mark {
  width: 34px; height: 34px; flex: none; border-radius: 50%; background: var(--bad-bg);
  color: var(--red); font-weight: 800; display: grid; place-items: center;
}
.err-title { margin: 0; font-weight: 700; }
.err-msg { margin: 2px 0 0; color: var(--sub); font-size: 14px; }

/* Summary + gauge */
.summary-head { display: flex; align-items: center; gap: 12px; margin-bottom: 16px; }
.pill { padding: 6px 14px; border-radius: 999px; font-size: 13px; font-weight: 700; }
.pill.ok { background: var(--ok-bg); color: var(--green); }
.pill.warn { background: var(--warn-bg); color: var(--amber); }
.pill.bad { background: var(--bad-bg); color: var(--red); }
.file-name { margin: 0; font-size: 13px; color: var(--sub); overflow: hidden; text-overflow: ell
  ipsis; white-space: nowrap; }

.summary-body { display: flex; align-items: center; gap: 24px; }
.gauge { position: relative; width: 120px; height: 120px; flex: none; }
.donut { width: 120px; height: 120px; }
.donut-track { fill: none; stroke: var(--line); stroke-width: 12; }
.donut-arc {
  fill: none; stroke: var(--blue); stroke-width: 12; stroke-linecap: round;
  transform: rotate(-90deg); transform-origin: center;
  transition: stroke-dashoffset .9s cubic-bezier(.22, .61, .36, 1);
}
.gauge-center { position: absolute; inset: 0; display: grid; place-items: center; }
.gauge-num { font-size: 34px; font-weight: 800; line-height: 1; font-variant-numeric: tabular-nu
  ms; }
.gauge-cap { font-size: 11px; color: var(--sub); letter-spacing: 1px; }

.metrics { display: flex; flex-direction: column; gap: 12px; }
.metric { display: flex; flex-direction: column; gap: 2px; }
.metric-label { font-size: 12px; color: var(--sub); }
.tip { cursor: help; text-decoration: underline dotted; text-underline-offset: 3px;
  text-decoration-color: color-mix(in srgb, var(--sub) 45%, transparent); }
.metric-val { font-size: 20px; font-weight: 700; font-variant-numeric: tabular-nums; }

/* Card head with actions: spacing lives on the CONTAINER so the title and the
   action stay vertically centered on the same line */
.card-head { display: flex; align-items: center; justify-content: space-between;
  gap: 12px; margin-bottom: 12px; }
.card-head .card-title { margin-bottom: 0; }
.btn.small { width: auto; margin-top: 0; padding: 8px 14px; font-size: 13px; border-radius: 10px
  ; }
.sel.small {
  font: inherit; font-size: 13px; font-weight: 600; padding: 7px 10px; border-radius: 10px;
```

```
  border: 1px solid var(--border); background: var(--field); color: var(--ink);
}
.play-row { display: flex; gap: 8px; align-items: center; }
.hint { margin: 8px 0 0; font-size: 12px; color: var(--sub); }

/* Spectrum + waterfall */
.spec { width: 100%; height: 120px; display: block; border-radius: 12px 12px 0 0; background: #0
  d1320; }
.fall { width: 100%; height: 150px; display: block; border-radius: 0 0 12px 12px; background: #0
  d1320; }

/* Constellation + playback */
.const { width: 100%; aspect-ratio: 1; border-radius: 14px; display: block; background: #0d1320
  ; }
.scrub { width: 100%; margin-top: 12px; accent-color: var(--blue); }

/* Batch table */
.table-wrap { overflow-x: auto; }
table.batch { width: 100%; border-collapse: collapse; font-size: 13.5px; }
table.batch th {
  text-align: left; font-size: 12px; color: var(--sub); font-weight: 600;
  padding: 8px 10px; border-bottom: 1px solid var(--line);
}
table.batch td { padding: 11px 10px; border-bottom: 1px solid var(--line);
  font-variant-numeric: tabular-nums; }
table.batch .pill { padding: 3px 10px; font-size: 12px; }
table.batch tbody tr { cursor: pointer; transition: background .12s; }
table.batch tbody tr:hover { background: var(--hover); }
table.batch .fn { font-weight: 600; max-width: 210px; overflow: hidden;
  text-overflow: ellipsis; white-space: nowrap; }

/* Param grid */
.param-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-top: 16px; }
.param { padding: 16px; margin: 0; animation: rise .4s ease both; }
.param-label { font-size: 12px; color: var(--sub); }
.param-val { font-size: 22px; font-weight: 800; margin-top: 4px; letter-spacing: -.3px;
  font-variant-numeric: tabular-nums; }
.param-note { font-size: 11px; color: var(--sub); margin-top: 2px; }
.chips { display: flex; flex-wrap: wrap; gap: 6px; margin-top: 10px; }
.chips:empty { display: none; }   /* no flags -> no ghost margin */
.chip { font-size: 11px; font-weight: 700; padding: 4px 9px; border-radius: 999px; }
.chip.hit { background: var(--ok-bg); color: var(--green); }
.chip.miss { background: var(--bad-bg); color: var(--red); }
.chip.warn { background: var(--warn-bg); color: var(--amber); }

/* Burst selector (chips that act) */
.chip-btn {
  font: inherit; font-size: 11px; font-weight: 700; padding: 4px 10px;
  border-radius: 999px; border: 1px solid var(--border); background: var(--field);
  color: var(--sub); cursor: pointer; transition: color .12s, border-color .12s;
}
.chip-btn:hover { border-color: var(--blue); color: var(--blue); }
.chip-btn.on { background: var(--blue-weak); color: var(--blue); border-color: transparent; }

/* Downloads */
.dl-row { display: flex; gap: 10px; flex-wrap: wrap; }
.dl-row .btn { flex: 1; min-width: 120px; }

/* Burst map: whole-record waterfall + clickable burst boxes */
.wf-wrap { position: relative; }
```

```
.wf-wrap .fall { height: 230px; border-radius: 12px; }
.boxes { position: absolute; inset: 0; pointer-events: none; }
.box {
  position: absolute; box-sizing: border-box; border: 1px solid rgba(255, 255, 255, .55);
  border-radius: 0; background: rgba(255, 255, 255, .06); cursor: pointer;
  pointer-events: auto; min-width: 7px; min-height: 12px;
  transition: background .12s, box-shadow .12s;
}
.box:hover, .box:focus-visible {
  background: rgba(255, 255, 255, .18); box-shadow: 0 0 0 1px #fff; outline: none;
}
.box-lbl {   /* hidden by default (a spectrogram full of labels is unreadable); shown on hover */
  position: absolute; top: -17px; left: -2px; white-space: nowrap; font-size: 10px;
  font-weight: 700; color: #fff; background: rgba(13, 19, 32, .85); padding: 1px 6px;
  border-radius: 6px; pointer-events: none; display: none; z-index: 2;
}
.box:hover .box-lbl, .box:focus-visible .box-lbl, .box.sel .box-lbl { display: block; }
.box.sel { border-color: #fff; box-shadow: inset 0 0 0 1px #fff; }
.pill.gray { background: var(--line); color: var(--sub); }
table.batch .fc { font-weight: 600; font-variant-numeric: tabular-nums; }

/* Non-digital emitter inline info */

@media (max-width: 420px) {
  .form-row { grid-template-columns: 1fr; }
  .param-grid { grid-template-columns: 1fr; }
  .summary-body { gap: 18px; }
}

@media (prefers-reduced-motion: reduce) {
  * { animation: none !important; transition: none !important; }
}
```

signus/web/app.js +84 −162 · 코드북 51쪽 · 새로 쓸 줄 8, 115, 132, 153, 155, 158-162, 228, 250, 253-256, 258-259, 262, 264-265, 283-285, 297, 320-321, 3

```
"use strict";
const $ = (id) => document.getElementById(id);
const API = "/api/analyze";
const FS_RE = /(?:^|[._])fs(\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:[._]|$)/i;
const RF_RE = /(?:^|[._])rf(\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:[._]|$)/i;
const REDUCED = matchMedia("(prefers-reduced-motion: reduce)").matches;
const state = { file: null, sidecar: null, meta: null, resp: null, batch: [],
                burst: null, overview: null };

/* --- filename parsing (mirror of sigio.parse_name; read-necessities only) --- */
const DTYPES = ["i8", "u8", "i16", "u16", "f32", "f64"];
// baudline-style aliases: <bits>t = two's complement (signed), <bits>o = offset (unsigned)
const DTYPE_ALIAS = { "8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "f64" };
const FMTS = { cplx: "iq", real: "real", iq: "iq" };   // iq = 옛 밑줄 형식의 이름
/* <이름>.cplx|real.<샘플레이트>.<16t>.pcm — 샘플레이트는 접두사가 없으므로 포맷 바로
   다음 자리로 찾는다. 옛 밑줄 형식(<이름>_fs…_iq_i16.iq)도 그대로 읽는다. sigio.py 와 규칙이
   같아야 한다: 여기서 갈리면 웹은 되고 CLI 는 안 되는(또는 그 반대) 조용한 불일치가 된다. */
function parseName(name) {
  const base = name.replace(/^.*\//, "").toLowerCase();
  const mt = FS_RE.exec(base), rt = RF_RE.exec(base), toks = base.split(/[._]/);
  // 진짜 포맷 토막 = "숫자 + 아는 제원 토막"을 데리고 다니는 마지막 후보. 라벨의 real/cplx/
  // u8/be 오염과 점 낀 샘플레이트(2.4e6 -> 2 Hz)를 같은 관문으로 막는다 (sigio.py 와 동일 규칙)
  .
  const spec = (t) => DTYPES.includes(t) || t in DTYPE_ALIAS || t === "be" || t === "bitrev"
```

```
25 24        || t === "pcm" || t.startsWith("rf");
26 25    const hit = (k) => /^\d+$/.test(toks[k + 1] || "") && spec(toks[k + 2] || "");
27 26    const cands = toks.map((t, k) => (t in FMTS ? k : -1)).filter((k) => k >= 0);
28 27    const hits = cands.filter(hit);
29 28    const i = hits.length ? hits[hits.length - 1] : (cands.length ? cands[0] : -1);
30 29    const tail = i >= 0 ? toks.slice(i + 1) : toks;   // 제원은 포맷 토막 뒤에만 산다
31 30    const dt = tail.find((t) => DTYPES.includes(t) || t in DTYPE_ALIAS);
32 31    return {
33 32      fs: hits.includes(i) ? parseFloat(toks[i + 1]) : (mt ? parseFloat(mt[1]) : null),
34 33      fmt: i >= 0 ? FMTS[toks[i]] : null,
35 34      dtype: dt ? (DTYPE_ALIAS[dt] || dt) : "i16",
36 35      endian: tail.includes("be") ? "be" : "le",
37 36      bitrev: tail.includes("bitrev"),
38 37      rf: rt ? parseFloat(rt[1]) : null,
39 38    };
40 39  }
41 40  /* SigMF: core:datatype -> our fmt/dtype/endian */
42 41  const SIGMF = { cf32: ["iq", "f32"], ci16: ["iq", "i16"], ci8: ["iq", "i8"],
43 42          cu8: ["iq", "u8"], rf32: ["real", "f32"], ri16: ["real", "i16"] };
44 43  function metaFromSigmf(doc) {
45 44    const g = doc && doc.global;
46 45    if (!g || !g["core:datatype"]) return null;
47 46    const dt = g["core:datatype"], base = dt.replace(/_[lb]e$/, "");
48 47    if (!(base in SIGMF)) return null;
49 48    const [fmt, dtype] = SIGMF[base];
50 49    const cap = (doc.captures || [])[0] || {};
51 50    return { fs: Number(g["core:sample_rate"]), fmt, dtype,
52 51        endian: dt.endsWith("_be") ? "be" : "le", bitrev: false,
53 52        rf: cap["core:frequency"] != null ? Number(cap["core:frequency"]) : null };
54 53  }
55 54
56 55  /* --- number formatting --- */
57 56  function sig(x) {
58 57    const a = Math.abs(x);
59 58    return parseFloat(a >= 100 ? x.toFixed(0) : a >= 10 ? x.toFixed(1) : x.toFixed(2)).toString();
60 59  }
61 60  function fmtHz(v) {
62 61    if (v == null) return "—";
63 62    const a = Math.abs(v);
64 63    if (a >= 1e6) return sig(v / 1e6) + " MHz";
65 64    if (a >= 1e3) return sig(v / 1e3) + " kHz";
66 65    return sig(v) + " Hz";
67 66  }
68 67  function fmtRf(v) {   // absolute RF: keep kHz resolution even in the 100s-of-MHz range
69 68    return (v / 1e6).toFixed(3) + " MHz";
70 69  }
71 70
72 71  /* --- ideal constellation points (mirror of constellations.ideal_points) --- */
73 72  function idealPoints(mod) {
74 73    if (mod === "bpsk") return [{ i: 1, q: 0 }, { i: -1, q: 0 }];
75 74    if (mod === "qpsk" || mod === "8psk") {
76 75      const m = mod === "qpsk" ? 4 : 8, off = mod === "qpsk" ? Math.PI / 4 : 0, p = [];
77 76      for (let k = 0; k < m; k++) p.push({ i: Math.cos(off + 2 * Math.PI * k / m), q: Math.sin(of
   ↳ ↳      f + 2 * Math.PI * k / m) });
78 77      return p;
79 78    }
80 79    const n = mod === "16qam" ? 4 : mod === "32qam" ? 6 : 8, lv = [], p = [];
81 80    for (let k = 0; k < n; k++) lv.push(k * 2 - (n - 1));
82 81    for (const qi of lv) for (const ii of lv) {
83 82      if (mod === "32qam" && Math.abs(ii * qi) === 25) continue;  // cross: corners removed
```

```
    p.push({ i: ii, q: qi });
  }
  let e = 0;
  for (const pt of p) e += pt.i * pt.i + pt.q * pt.q;
  const norm = Math.sqrt(e / p.length);
  return p.map((pt) => ({ i: pt.i / norm, q: pt.q / norm }));
}

/* --- file intake: one data file (+sidecar) or many (batch) --- */
async function acceptFiles(list) {
  const files = [...list];
  const metas = files.filter((f) => /\.(json|sigmf-meta)$/i.test(f.name));
  const data = files.filter((f) => !metas.includes(f));
  if (!data.length) return showError("데이터 파일을 찾을 수 없어요.");
  if (data.length > 1) return runBatch(data, metas);

  state.file = data[0];
  state.resp = null;
  state.batch = [];
  state.burst = null;
  state.overview = null;
  // 사이드카는 이름이 같은 캡처의 것만 쓴다 -- 확장자만 보고 집으면 다른 신호의 fs/fmt 로
  // 조용히 읽는다 (일괄 모드는 이미 stem 대조를 한다: 두 모드가 다르면 그게 또 함정이다)
  const mine = (m, ext) => {
    const stem = m.name.toLowerCase().slice(0, -ext.length);
    const dn = state.file.name.toLowerCase();
    return dn.startsWith(stem) && (dn.length === stem.length || dn[stem.length] === ".");
  };
  const sm = metas.find((m) => m.name.toLowerCase().endsWith(".sigmf-meta") && mine(m, ".sigmf-m
    eta"));
  const truth = metas.find((m) => m.name.toLowerCase().endsWith(".json") && mine(m, ".json"));
  state.sidecar = truth ? await readJson(truth) : null;   // client-side only, never sent
  state.meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(state.file.name);
  if (state.meta && state.meta.fs && state.meta.fmt) runFirst();
  else showMetaForm();
}
const readJson = (f) => f.text().then(JSON.parse).catch(() => null);

function showMetaForm() {
  hideAll();
  state.meta = state.meta || {};
  if (state.meta.fs) $("mFs").value = state.meta.fs;
  $("mFmt").value = state.meta.fmt || "";
  $("mDtype").value = state.meta.dtype || "i16";
  show($("metaForm"));
}
$("metaGo").onclick = () => {
  const fs = parseFloat($("mFs").value), fmt = $("mFmt").value;
  if (!(fs > 0) || !fmt) return showError("샘플레이트와 포맷을 입력해주세요.");
  state.meta = { fs, fmt, dtype: $("mDtype").value, endian: "le", bitrev: false };
  runFirst();
};

/* --- server call (contract-frozen) --- */
function query(file, m, burst) {
  const q = new URLSearchParams({ name: file.name });
  if (m.fs) q.set("fs", m.fs);
  if (m.fmt) q.set("fmt", m.fmt);
  if (m.dtype) q.set("dtype", m.dtype);
  if (m.endian) q.set("endian", m.endian);
```

```
145 142   if (m.bitrev) q.set("bitrev", "1");
146 143   if (m.rf != null) q.set("rf", m.rf);
147 144   if (burst != null) q.set("burst", burst);
148 145   return q;
149 146 }
150 147 async function postTo(api, file, m, burst) {
151 148   const res = await fetch(api + "?" + query(file, m, burst), { method: "POST", body: file });
152 149   const data = await res.json();
153 150   if (!res.ok || data.error) throw new Error(data.error || `서버 오류 (${res.status})`);
154 151   return data;
155 152 }
    153 + async function runFirst() {                    // entry: first analyze of a fresh capture
157 154   hideAll();
    155 +   $("loadMsg").textContent = "신호를 분석하고 있어요…";
160 156   show($("loading"));
161 157   try {
    158 +     const resp = await postTo(API, state.file, state.meta);
    159 +     state.overview = resp.overview || null;
    160 +     state.resp = resp;
    161 +     render(resp);
    162 +     if (state.overview) renderBurstMap(state.overview, resp);
168 163   } catch (e) {
169 164     showError(e.message);
170 165   }
171 166 }
172 167 async function analyze() {                      // burst re-selection within a single signal
173 168   hideAll();
174 169   $("loadMsg").textContent = "신호를 분석하고 있어요…";
175 170   show($("loading"));
176 171   try {
177 172     state.resp = await postTo(API, state.file, state.meta, state.burst);
178 173     render(state.resp);
179 174     if (state.overview) renderBurstMap(state.overview, state.resp);
180 175     $("backBatch").classList.toggle("hidden", !state.batch.length);
181 176   } catch (e) {
182 177     showError(e.message);
183 178   }
184 179 }
185 180
186 181 /* --- batch mode --- */
187 182 async function runBatch(data, metas) {
188 183   hideAll();
189 184   show($("loading"));
190 185   const truths = new Map(metas.filter((m) => /\.json$/i.test(m.name))
191 186     .map((m) => [m.name.replace(/\.json$/i, ""), m]));
192 187   const sigmfs = new Map(metas.filter((m) => /\.sigmf-meta$/i.test(m.name))
193 188     .map((m) => [m.name.replace(/\.sigmf-meta$/i, ""), m]));
194 189   const rows = [];
195 190   for (let i = 0; i < data.length; i++) {
196 191     const f = data[i];
197 192     $("loadMsg").textContent = `일괄 분석 중… (${i + 1}/${data.length}) ${f.name}`;
198 193     // same meta precedence as single-file mode: a .sigmf-meta sidecar (matched by stem)
199 194     // beats filename tokens
200 195     const sm = sigmfs.get(f.name.replace(/\.[^.]+$/, ""));
201 196     const meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(f.name);
202 197     let resp = null, err = null;
203 198     try {
204 199       if (!meta.fs || !meta.fmt) throw new Error("파일명에 fs/포맷 정보가 없어요");
205 200       resp = await postTo(API, f, meta);
206 201     } catch (e) { err = e.message; }
```

```
207 202     const tf = truths.get(f.name);
208 203     // meta is stored per row: a later burst-chip re-analyze posts with THIS file's meta --
209 204     // it used to read state.meta, which a fresh batch never set (crash) and a previous
210 205     // single-file run left stale (silent wrong-fs decode)
211 206     rows.push({ file: f, meta, resp, err, sidecar: tf ? await readJson(tf) : null });
212 207   }
213 208   state.batch = rows;
214 209   hideAll();
215 210   renderBatch(rows);
216 211 }
217 212
218 213 function renderBatch(rows) {
219 214   $("batchBody").innerHTML = rows.map((r, i) => {
220 215     if (r.err) return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
221 216       `<td colspan="4" style="color:var(--red)">${r.err}</td></tr>`;
222 217     const d = r.resp.detected, lock = Math.round(r.resp.quality.lock);
223 218     const cls = lock >= 60 ? "ok" : lock >= 40 ? "warn" : "bad";
224 219     return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
225 220       `<td>${d.mod.toUpperCase()}</td><td>${fmtHz(d.fc)}</td><td>${fmtHz(d.baud)}</td>` +
226 221       `<td><span class="pill ${cls}">${lock}</span></td></tr>`;
227 222   }).join("");
228 223   $("batchBody").querySelectorAll("tr").forEach((tr) => {
229 224     tr.onclick = () => {
230 225       const r = state.batch[+tr.dataset.i];
231 226       if (!r.resp) return;
232 227       state.file = r.file; state.meta = r.meta; state.resp = r.resp; state.sidecar = r.sidecar;
228 +       state.burst = null; state.overview = null;
234 229       hideAll();
235 230       render(r.resp);
236 231       const b = $("backBatch");
237 232       b.textContent = "← 목록으로";
238 233       b.onclick = () => { hideAll(); renderBatch(state.batch); };
239 234       b.classList.remove("hidden");
240 235     };
241 236   });
242 237   show($("batchCard"));
243 238 }
244 239 $("backBatch").onclick = () => { hideAll(); renderBatch(state.batch); };
245 240 $("dlCsv").onclick = () => {
246 241   const head = "file,mod,fc_hz,baud_hz,rolloff,lock,mer_db,snr_est_db\n";
247 242   const body = state.batch.filter((r) => r.resp).map((r) => {
248 243     const d = r.resp.detected, q = r.resp.quality;
249 244     return [r.file.name, d.mod, d.fc, d.baud, d.rolloff ?? "", q.lock, q.mer_db,
250 245             r.resp.snr_est_db ?? ""].join(",");
251 246   }).join("\n");
252 247   saveBlob("signus_batch.csv", head + body, "text/csv");
253 248 };
254 249
250 +   /* --- burst map: whole-record waterfall with clickable time-burst boxes --- */
368 251 function renderBurstMap(ov, result) {
369 252   show($("burstMap"));
253 +   const n = ov.n || 1, bs = result.bursts || [];
254 +   paintWaterfall($("burstFall"), ov.waterfall, 150,
255 +                  bs.map((b, i) => [b.start / n, b.end / n, String(i + 1)]));
256 +   const host = $("burstBoxes");
372 257   host.innerHTML = "";
258 +   bs.forEach((b, i) => {
259 +     const left = Math.max(0, b.start / n * 100);
375 260     const dur = (b.end - b.start) / ov.fs;
376 261     const lbl = `버스트 ${i + 1} · ${dur >= 1 ? dur.toFixed(2) + " s" : (dur * 1e3).toFixed(0
```

```
      ) + " ms"}`;
+     const box = document.createElement("button");   // full height (one signal), spans t0..t1 i
+       n time
    box.className = "box" + (i === result.burst_idx ? " sel" : "");
+     box.style.top = "0%"; box.style.height = "100%"; box.style.left = left + "%";
+     box.style.width = Math.min(100 - left, Math.max(0.8, (b.end - b.start) / n * 100)) + "%";
    box.title = lbl;
    box.innerHTML = `<span class="box-lbl">${lbl}</span>`;
    box.onclick = () => { state.burst = i; analyze(); };   // re-analyse that burst; map persist
      s
    host.appendChild(box);
  });
}

/* --- rendering --- */
function statusOf(lock) {
  if (lock >= 60) return ["복조 성공", "ok"];
  if (lock >= 40) return ["부분 복조", "warn"];
  return ["복조 실패", "bad"];
}
function render(d) {
  hideAll();
  show($("results"));
  $("burstMap").classList.add("hidden");   // re-shown by renderBurstMap only in multi-burst sin
    gle mode
+   const lock = d.quality.lock;
+   let [txt, cls] = statusOf(lock);
+   if (d.detected.chirp) { txt = "처프 특성 보고"; cls = "warn"; }
  const pill = $("statusPill");
  pill.textContent = txt;
  pill.className = "pill " + cls;
  $("fileName").textContent =
    `${state.file.name} · ${fmtHz(d.fs)} · ${d.fmt === "real" ? "Real PCM" : "IQ"}`;
  $("merVal").textContent = d.quality.mer_db == null ? "—" : d.quality.mer_db.toFixed(1) + " dB"
    ;
  $("evmVal").textContent = d.quality.evm == null ? "—" : (d.quality.evm * 100).toFixed(1) + " %
    ";
  $("snrVal").textContent = snrText(d);

  const flags = [];
  if (d.family === "fsk") flags.push('<span class="chip warn">FSK 계열 · 주파수 판별</span>');
+   if (d.family === "chirp") flags.push('<span class="chip warn">처프/CSS · 특성 판독</span>');
  if (d.eq && d.eq.applied) flags.push(`<span class="chip hit">등화기 적용 · ${
    d.eq.mode === "fse" ? "T/2 분수간격" : "심볼간격"}</span>`);
  if (d.detected.alias_resolved) flags.push('<span class="chip warn">반송파 앨리어스 보정</span>
    ');
  if (d.detected.baud_fallback) flags.push('<span class="chip warn">심볼레이트 대역폭 폴백</span
    >');
  if (d.detected.carrier_ambiguous) flags.push('<span class="chip warn">반송파 모호 · 앨리어
    싱 가능</span>');
  $("flagRow").innerHTML = flags.join("");

  renderBursts(d);
  animateGauge(lock, cls);
  renderParams(d);
  drawSpectrum(d);
  drawWaterfall(d);
  startPlay(d);
}
function snrText(d) {
```

```
425 313   const s = d.snr_est_db;
426 314   if (s == null || d.quality.lock < 30) return "—";
427 315   return (s > 28 ? "≥28" : s.toFixed(1)) + " dB";
428 316 }
429 317
430 318 function renderBursts(d) {
431 319   const row = $("burstRow"), bs = d.bursts || [];
    320 +   // when the burst MAP is shown (multi-burst capture) the map replaces the chips
    321 +   if (state.overview || bs.length < 2) { row.innerHTML = ""; return; }
436 322   const dur = (b) => {
437 323     const sec = (b.end - b.start) / d.fs;
438 324     return sec >= 1 ? sec.toFixed(2) + " s" : (sec * 1e3).toFixed(1) + " ms";
439 325   };
440 326   row.innerHTML = bs.map((b, i) =>
441 327     `<button class="chip-btn${i === d.burst_idx ? " on" : ""}" data-i="${i}">` +
442 328     `버스트 ${i + 1} · ${dur(b)}</button>`).join("");
443 329   row.querySelectorAll("button").forEach((el) => {
444 330     el.onclick = () => { state.burst = +el.dataset.i; analyze(); };
445 331   });
446 332 }
447 333
448 334 function chip(hit, truthTxt) {
449 335   return `<span class="chip ${hit ? "hit" : "miss"}">정답 ${truthTxt} · ${hit ? "일치" : "불일치
      "}</span>`;
450 336 }
451 337 function renderParams(d) {
452 338   const det = d.detected, t = state.sidecar && state.sidecar.truth;
453 339   const near = (a, b, tol) => Math.abs(a - b) <= tol;
    340 +   if (det.chirp) {              // 처프/CSS: 성상도 제원 대신 특성 (복조 없음, 판독만)
    341 +     const c = det.chirp;
    342 +     const crows = [
    343 +       ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc), null,
    344 +         det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
    345 +       ["처프", `${c.up ? "▲ 상승" : "▼ 하강"} · ${(c.mu / 1e9).toFixed(3)} MHz/ms`, null,
    346 +         `점유대역폭 ${fmtHz(c.bw)}`],
    347 +       ["변조방식", c.sf ? `LoRa 추정 SF${c.sf}` : "처프(FMCW/CSS)", null,
    348 +         c.sf ? `심볼레이트 ${fmtHz(c.rs)} · 심볼시간 ${(c.tsym * 1e3).toFixed(2)} ms` : ""],
    349 +     ];
    350 +     $("paramGrid").innerHTML = crows.map(paramCard).join("");
    351 +     return;
    352 +   }
454 353   const rows = [
455 354     ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc),
456 355       t && t.fc != null && chip(near(det.fc, t.fc, Math.max(300, 0.02 * t.baud)), fmtHz(t.fc)),
457 356       det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
458 357     ["심볼레이트", fmtHz(det.baud), t && t.baud != null && chip(near(det.baud, t.baud, 0.03 * t.
      baud), fmtHz(t.baud)),
459 358       det.baud_conf ? `스펙트럴 라인 강도 ×${Math.round(det.baud_conf)}` : ""],
460 359     ["변조방식", det.mod.toUpperCase(), t && t.mod != null && chip(det.mod === t.mod, t.mod.toUp
      perCase()),
461 360       det.symmetry ? `대칭 ${det.symmetry}` : "주파수 레벨"],
462 361   ];
463 362   if (d.family === "fsk") {
464 363     rows.push(["변조지수 h", det.h.toFixed(2),
465 364       t && t.h != null && chip(near(det.h, t.h, 0.15), t.h.toFixed(2)), ""]);
466 365   } else {
467 366     rows.push(["롤오프", det.rolloff.toFixed(2),
468 367       t && t.rolloff != null && chip(near(det.rolloff, t.rolloff, 0.11), t.rolloff.toFixed(2))
        , ""]);
469 368   }
```

```
+   $("paramGrid").innerHTML = rows.map(paramCard).join("");
+ }
+
+ function paramCard([label, val, hit, note]) {
+   return `
    <div class="card param">
      <div class="param-label">${label}</div>
      <div class="param-val">${val}</div>
      ${note ? `<div class="param-note">${note}</div>` : ""}
      ${hit ? `<div class="chips">${hit}</div>` : ""}
+   </div>`;
  }

  function animateGauge(lock, cls) {
    const arc = $("donutArc"), num = $("lockNum"), C = 2 * Math.PI * 52;
    arc.style.stroke = { ok: "#12b886", warn: "#f59f00", bad: "#fa5252" }[cls];
    arc.style.strokeDasharray = C;
    const target = C * (1 - Math.max(0, Math.min(100, lock)) / 100);
    if (REDUCED) { arc.style.strokeDashoffset = target; num.textContent = Math.round(lock); return ; }
    arc.style.strokeDashoffset = C;
    requestAnimationFrame(() => { arc.style.strokeDashoffset = target; });
    const t0 = performance.now();
    const step = (t) => {
      const k = Math.min(1, (t - t0) / 900);
      num.textContent = Math.round(lock * (1 - Math.pow(1 - k, 3)));
      if (k < 1) requestAnimationFrame(step);
    };
    requestAnimationFrame(step);
  }

  /* --- canvas helpers --- */
  function fit(cv, hCss) {
    const dpr = window.devicePixelRatio || 1, w = cv.clientWidth || 320;
    const h = hCss || w;
    cv.width = w * dpr; cv.height = h * dpr;
    const g = cv.getContext("2d");
    g.setTransform(dpr, 0, 0, dpr, 0, 0);
    return [g, w, h];
  }
  function pct(a, p) {
    const s = [...a].sort((x, y) => x - y);
    return s[Math.min(s.length - 1, Math.max(0, Math.round(p * (s.length - 1))))];
  }

  /* --- spectrum --- */
  function drawSpectrum(d) {
+   // sa 프로브의 power spectrum 판과 같은 조판: 흑배경, 백색 곡선, 회색(0.35) 점선 격자,
+   // 바닥 p5-2 .. 최대+2 dB. 검출된 중심주파수(실선)와 ±baud/2(점선)만 기능선으로 얹는다.
    const [g, w, h] = fit($("specCanvas"), 120);
+   g.fillStyle = "#000"; g.fillRect(0, 0, w, h);
    if (!d.spectrum) return;
    const f = d.spectrum.f, db = d.spectrum.db;
+   const lo = pct(db, 0.05) - 2, hi = pct(db, 1) + 2;
    const fmin = f[0], fmax = f[f.length - 1];
    const X = (v) => (v - fmin) / (fmax - fmin) * w;
    const Y = (v) => h - (Math.max(lo, Math.min(hi, v)) - lo) / (hi - lo) * (h - 8) - 4;
+   g.strokeStyle = "rgba(255,255,255,.35)"; g.setLineDash([1, 5]);
+   for (let k = 1; k <= 4; k++) {                 // fs/2 의 k/5 지점 세로 격자 (sa 와 동일)
+     const gx = fmin + (fmax - fmin) * k / 5;
```

```
    +     g.beginPath(); g.moveTo(X(gx), 0); g.lineTo(X(gx), h); g.stroke();
    +   }
520 430   const det = d.detected, half = det.baud / 2e3;
    +   g.setLineDash([4, 4]);
522 432   for (const gf of [det.fc / 1e3 - half, det.fc / 1e3 + half]) {
523 433     g.beginPath(); g.moveTo(X(gf), 0); g.lineTo(X(gf), h); g.stroke();
524 434   }
525 435   g.setLineDash([]);
    +   g.strokeStyle = "rgba(255,255,255,.6)";
527 437   g.beginPath(); g.moveTo(X(det.fc / 1e3), 0); g.lineTo(X(det.fc / 1e3), h); g.stroke();
    +   g.strokeStyle = "#ebebeb"; g.lineWidth = 1.2; g.beginPath();
529 439   f.forEach((v, k) => (k ? g.lineTo(X(v), Y(db[k])) : g.moveTo(X(v), Y(db[k]))));
530 440   g.stroke();
    +   g.fillStyle = "rgba(255,255,255,.6)"; g.font = "10px sans-serif";
532 442   g.fillText(sig(fmin) + " kHz", 4, h - 4);
533 443   const tmax = sig(fmax) + " kHz";
534 444   g.fillText(tmax, w - g.measureText(tmax).width - 4, h - 4);
535 445 }
536 446
537 447 /* --- waterfall --- */
538 448 function drawWaterfall(d) { paintWaterfall($("fallCanvas"), d.waterfall, 150); }
    + function paintWaterfall(canvas, wf, hCss, marks) {
    +   // sa 프로브의 PNG 와 같은 조판: 흑백(최대 235), 바닥 p25 + 대비폭 10..35dB 클램프,
    +   // 가로=시간, 위=+주파수, 보간 없는 픽셀. marks 가 오면 위 번호 레인 + 아래 검출 띠.
540 452   const [g, w, h] = fit(canvas, hCss);
    +   g.fillStyle = "#000"; g.fillRect(0, 0, w, h);
542 454   if (!wf || !wf.rows) return;
    +   const lo = pct(wf.db, 0.25);
    +   const rg = Math.max(10, Math.min(35, pct(wf.db, 0.995) - lo));
    +   const img = g.createImageData(wf.rows, wf.bins);          // x=시간(row), y=주파수(bin)
    +   for (let r = 0; r < wf.rows; r++) {
    +     for (let b = 0; b < wf.bins; b++) {
    +       const t = Math.max(0, Math.min(1, (wf.db[r * wf.bins + b] - lo) / rg));
    +       const o = ((wf.bins - 1 - b) * wf.rows + r) * 4;      // +fs/2 가 위
    +       img.data[o] = img.data[o + 1] = img.data[o + 2] = Math.round(t * 235);
    +       img.data[o + 3] = 255;
    +     }
552 465   }
    +   createImageBitmap(img).then((bmp) => {
    +     g.imageSmoothingEnabled = false;
    +     g.drawImage(bmp, 0, 0, w, h);
    +     if (!marks) return;
    +     g.fillStyle = "#ebebeb"; g.font = "bold 11px ui-monospace, monospace";
    +     marks.forEach(([a, b, num]) => {
    +       g.fillRect(a * w, h - 6, Math.max(2, (b - a) * w), 4);         // 검출 띠
    +       g.fillText(num, ((a + b) / 2) * w - g.measureText(num).width / 2, 12);  // 번호 레인
    +     });
    +   });
554 476 }
555 477
556 478 /* --- constellation playback (the headline) --- */
557 479 const play = { raf: 0, i: 0, on: false, data: null };
558 480 let CG = null;   // cached constellation geometry
559 481
560 482 function stopPlay() { cancelAnimationFrame(play.raf); play.raf = 0; play.on = false; }
561 483 function setPlayBtn(on) { $("playBtn").textContent = on ? "⏸ 일시정지" : "▶ 재생"; }
562 484
563 485 function startPlay(d) {
564 486   stopPlay();
565 487   play.data = d;
```

```
566 488   play.i = 0;
567 489   const n = d.constellation.i.length;
568 490   $("scrub").max = String(Math.max(1, n));
569 491   $("playInfo").textContent = `${n.toLocaleString()} 심볼 · 시간 순서로 재생 (잔광 효과)`;
570 492   drawConstBase(d);
571 493   if (REDUCED) {                        // static full view, no motion
572 494     drawPoints(d, 0, n);
573 495     $("scrub").value = String(n);
574 496     setPlayBtn(false);
575 497     return;
576 498   }
577 499   play.on = true;
578 500   setPlayBtn(true);
579 501   play.raf = requestAnimationFrame(loop);
580 502 }
581 503 $("playBtn").onclick = () => {
582 504   if (!play.data) return;
583 505   if (play.on) { stopPlay(); setPlayBtn(false); }
584 506   else { play.on = true; setPlayBtn(true); play.raf = requestAnimationFrame(loop); }
585 507 };
586 508 $("scrub").oninput = () => {
587 509   if (!play.data) return;
588 510   stopPlay(); setPlayBtn(false);
589 511   play.i = +$("scrub").value;
590 512   drawConstBase(play.data);
591 513   drawPoints(play.data, 0, play.i);    // redraw history up to the scrub point
592 514 };
593 515
594 516 function loop() {
595 517   const d = play.data, n = d.constellation.i.length;
596 518   const step = Math.max(1, Math.round(n / 600) * (+$("speedSel").value));
597 519   const to = Math.min(n, play.i + step);
598 520   fade();                               // phosphor persistence
599 521   drawPoints(d, play.i, to);
600 522   play.i = to >= n ? 0 : to;
601 523   if (play.i === 0) drawConstBase(d);  // loop: clear the trail
602 524   $("scrub").value = String(play.i);
603 525   if (play.on) play.raf = requestAnimationFrame(loop);
604 526 }
605 527
606 528 function drawConstBase(d) {
607 529   const [g, w] = fit($("constCanvas"));
608 530   const fsk = d.family === "fsk";
609 531   const I = d.constellation.i, Q = d.constellation.q;
610 532   const ideal = fsk ? [] : idealPoints(d.detected.mod);
611 533   let lim = 0;
612 534   for (let k = 0; k < I.length; k++) lim = Math.max(lim, Math.abs(I[k]), Math.abs(Q[k]));
613 535   for (const p of ideal) lim = Math.max(lim, Math.abs(p.i), Math.abs(p.q));
614 536   lim = (lim || 1) * 1.12;
615 537   const R = w / 2 * 0.9;
616 538   CG = { g, w, lim, R, map: (re, im) => [w / 2 + re / lim * R, w / 2 - im / lim * R] };
617 539
618 540   g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, w);
619 541   g.strokeStyle = "rgba(255,255,255,.06)"; g.lineWidth = 1;
620 542   for (let f = -1; f <= 1; f += 0.5) {
621 543     const [gx] = CG.map(f, 0), [, gy] = CG.map(0, f);
622 544     g.beginPath(); g.moveTo(gx, 0); g.lineTo(gx, w); g.stroke();
623 545     g.beginPath(); g.moveTo(0, gy); g.lineTo(w, gy); g.stroke();
624 546   }
625 547   if (fsk) {                            // frequency levels as dashed bands
```

```
    g.strokeStyle = "rgba(255,255,255,.5)"; g.setLineDash([6, 5]);
    for (const lv of [...new Set(I.map((v) => Math.round(v)))].filter((v) => Math.abs(v) <= 8)
      ) {
      const [, yy] = CG.map(0, lv);
      g.beginPath(); g.moveTo(0, yy); g.lineTo(w, yy); g.stroke();
    }
    g.setLineDash([]);
  } else {
    g.strokeStyle = "rgba(255,255,255,.75)"; g.lineWidth = 1.4;
    for (const p of ideal) {
      const [x, y] = CG.map(p.i, p.q);
      g.beginPath(); g.arc(x, y, 6, 0, 7); g.stroke();
    }
  }
}
function fade() {
  if (!CG) return;
  CG.g.fillStyle = "rgba(13,19,32,.14)";   // translucent wash = phosphor decay
  CG.g.fillRect(0, 0, CG.w, CG.w);
}
function drawPoints(d, from, to) {
  if (!CG || to <= from) return;
  const g = CG.g, I = d.constellation.i, Q = d.constellation.q;
  const fsk = d.family === "fsk";
  const a = Math.max(0.08, Math.min(0.55, 150 / (to - from)));
  g.globalCompositeOperation = "lighter";
  g.fillStyle = `rgba(90,170,255,${a})`;
  for (let k = from; k < to; k++) {
    // FSK has no I/Q plane: spread levels vertically, jitter horizontally for density
    const [x, y] = fsk ? CG.map((k % 89) / 89 - 0.5, I[k]) : CG.map(I[k], Q[k]);
    g.beginPath(); g.arc(x, y, 1.8, 0, 7); g.fill();
  }
  g.globalCompositeOperation = "source-over";
}

/* --- downloads --- */
function saveBlob(name, text, type) {
  const url = URL.createObjectURL(new Blob([text], { type }));
  const a = document.createElement("a");
  a.href = url; a.download = name; a.click();
  URL.revokeObjectURL(url);
}
const stem = () => state.file.name.replace(/\.[^.]*$/, "");
$("copyBits").onclick = async (e) => {   // decoded bitstream -> clipboard, with brief feedback
  const btn = e.currentTarget, was = btn.textContent;
  try { await navigator.clipboard.writeText(state.resp.bits); btn.textContent = "복사됨 ✓"; }
  catch { btn.textContent = "복사 실패"; }
  setTimeout(() => { btn.textContent = was; }, 1400);
};
$("dlBits").onclick = () => saveBlob(stem() + ".bits.txt", state.resp.bits, "text/plain");
$("dlSyms").onclick = () => {
  const c = state.resp.constellation;
  saveBlob(stem() + ".symbols.csv", "i,q\n" + c.i.map((v, k) => `${v},${c.q[k]}`).join("\n"), "t
    ext/csv");
};
$("dlReport").onclick = () =>
  saveBlob(stem() + ".report.json", JSON.stringify(state.resp, null, 2), "application/json");

/* --- view helpers --- */
function show(el) { el.classList.remove("hidden"); }
```

```
684 606 function hideAll() {
685 607   stopPlay();
    608 +   ["metaForm", "loading", "errorCard", "results", "batchCard"]
687 609     .forEach((id) => $(id).classList.add("hidden"));
688 610   $("backBatch").classList.add("hidden");
689 611 }
690 612 function showError(msg) { hideAll(); $("errMsg").textContent = msg; show($("errorCard")); }
691 613
692 614 /* --- dropzone wiring --- */
693 615 const drop = $("dropCard"), input = $("fileInput");
694 616 // reset value first: re-picking the SAME file fires no change event otherwise
695 617 const pick = () => { input.value = ""; input.click(); };
696 618 drop.onclick = pick;
697 619 drop.onkeydown = (e) => { if (e.key === "Enter" || e.key === " ") { e.preventDefault(); pick()
  ↳   ↳   ; } };
698 620 input.onchange = () => { if (input.files.length) acceptFiles(input.files); };
699 621 drop.addEventListener("dragover", (e) => { e.preventDefault(); drop.classList.add("drag"); });
700 622 drop.addEventListener("dragleave", () => drop.classList.remove("drag"));
701 623 drop.addEventListener("drop", (e) => {
702 624   e.preventDefault(); drop.classList.remove("drag");
703 625   if (e.dataTransfer.files.length) acceptFiles(e.dataTransfer.files);
704 626 });
```